

# Bitcoins and Cryptocurrencies

***Note: most of the slides used in this course are derived from those available for the book “Bitcoins and Cryptocurrencies Technologies – A Comprehensive Introduction”, Arvind Narayanan, Joseph Bonneau, Edward Felten, Andrew Miller & Steven Goldfeder, 2016, Princeton University Press.***

# Mechanics of Bitcoin

## Recap: Bitcoin consensus

Bitcoin consensus gives us:

- Append-only ledger
- Decentralized consensus protocol
- Miners to validate transactions

*Assuming a currency exists to motivate miners!*

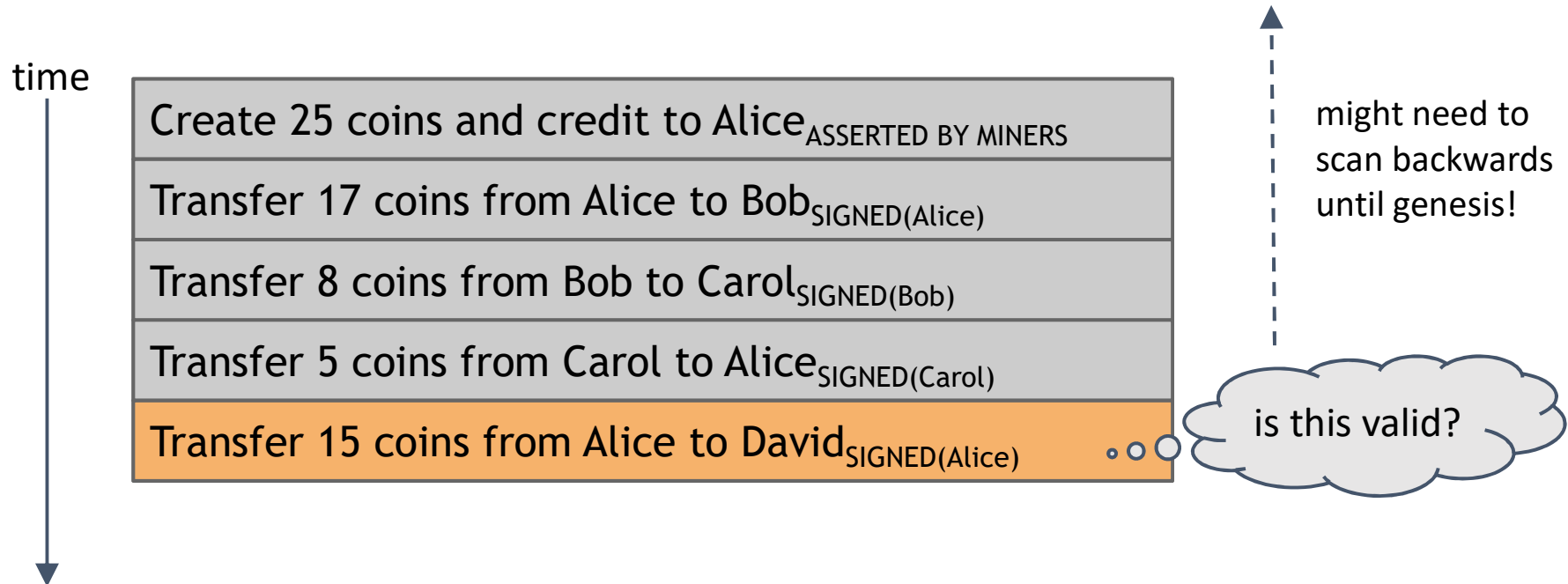
*In this chapter we will see how such a currency can be engineered*

# Bitcoin transactions

## An account-based ledger (*not* Bitcoin)

- Instead of blocks, **let's suppose individual transactions are added to the ledger one at a time.**
- You can **add some transactions** that **create new coins** and **credit them to somebody.**
- **And then later you can transfer the coins.**
- **That's all the information in the ledger about the transaction.**

# An account-based ledger (*not* Bitcoin)



SIMPLIFICATION: only one transaction per block

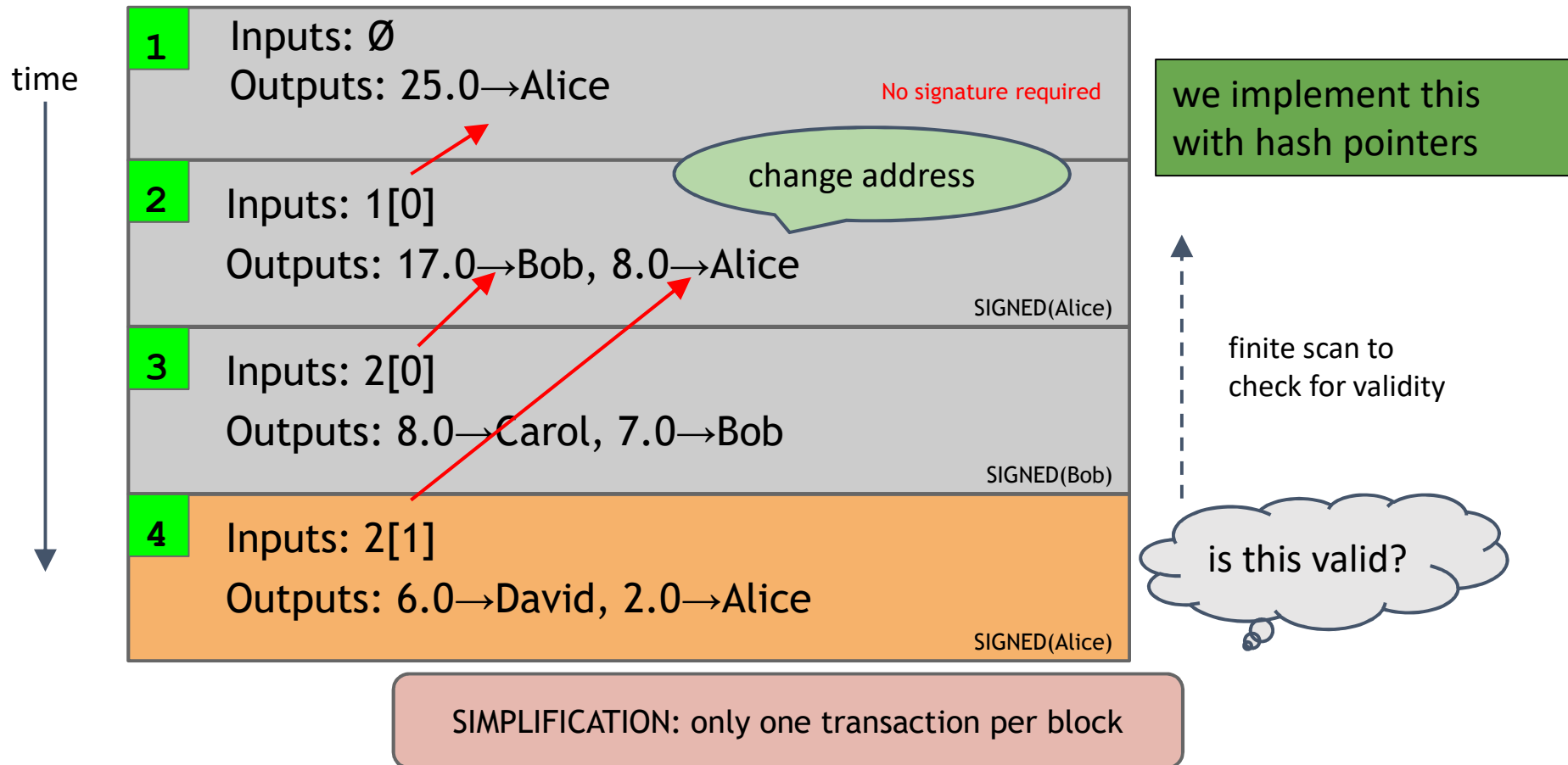
## An account-based ledger (*not* Bitcoin)

- The **downside** to this way of doing things is that **anyone who wants to determine whether a transaction is valid** will have to **keep track of these account balances**.
- **Because of these drawbacks, Bitcoin doesn't use an *account-based model*.**
- **Instead, Bitcoin uses a *ledger* that just keeps track of transactions, similar to *Scroogecoin*.**

## A transaction-based ledger (Bitcoin)

- **Transactions specify** a *number of inputs* and a *number of outputs* (recall PayCoins in ScroogeCoin).
- You can think of the *inputs as coins* being *consumed* (**created in a previous transaction**) and the *outputs as coins* being **created**.
- For **transactions** in *which new currency is being minted*, **no coins are being consumed** (recall CreateCoins in ScroogeCoin).
- **Each transaction** has a *unique identifier*.
- **Outputs are indexed** beginning *with 0*, so we refer to *the first output as “output 0.”*

# A transaction-based ledger (Bitcoin)



# A transaction-based ledger (Bitcoin)

- ***Change addresses:***

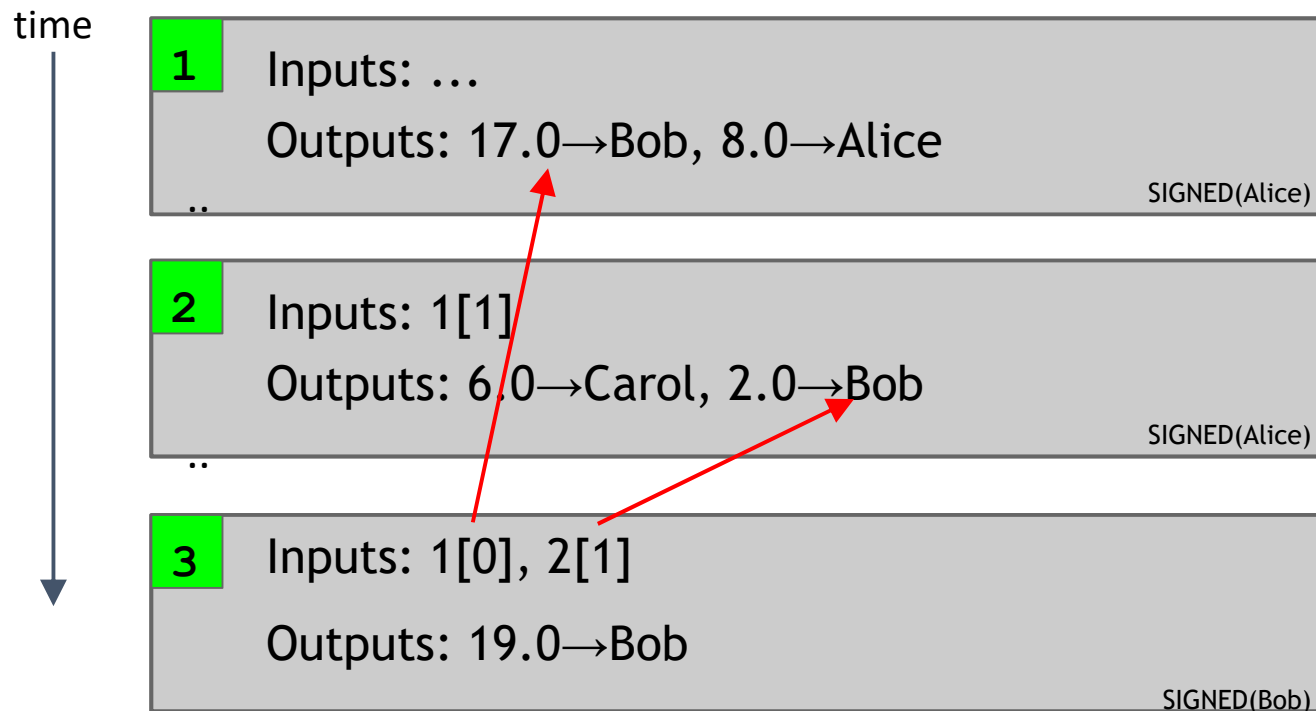
- Just as coins in Scroogecoin are *immutable, in Bitcoin*, either *all or none of a transaction output must be consumed* by **another transaction**.
- Alice only wants to pay 17 bitcoins to Bob, but the output that she owns is worth 25 bitcoins.
- So she needs to create a new output, where 8 bitcoins are sent back to herself.
- It could be a **different address from the one that owned the 25 bitcoins**, but it would have to be **owned by Alice**. This is called a **change address**.

# A transaction-based ledger (Bitcoin)

- ***Efficient verification:***

- When a new transaction is added to the ledger, how easy is it to check whether it is valid?
- In this example, we need to look up the transaction output that Alice referenced, make sure that it has a value of 25 bitcoins, and that it hasn't already been spent.
- Looking up the transaction output is easy, since we're using hash pointers.
- To ensure it hasn't been spent, we need to scan the block chain between the *referenced transaction* and the *latest block*.
- We **don't need to go all the way back to the beginning of the block chain**, and it **doesn't require keeping any additional data structures** (although, as we'll see, *additional data structures will speed things up*).

# Merging value



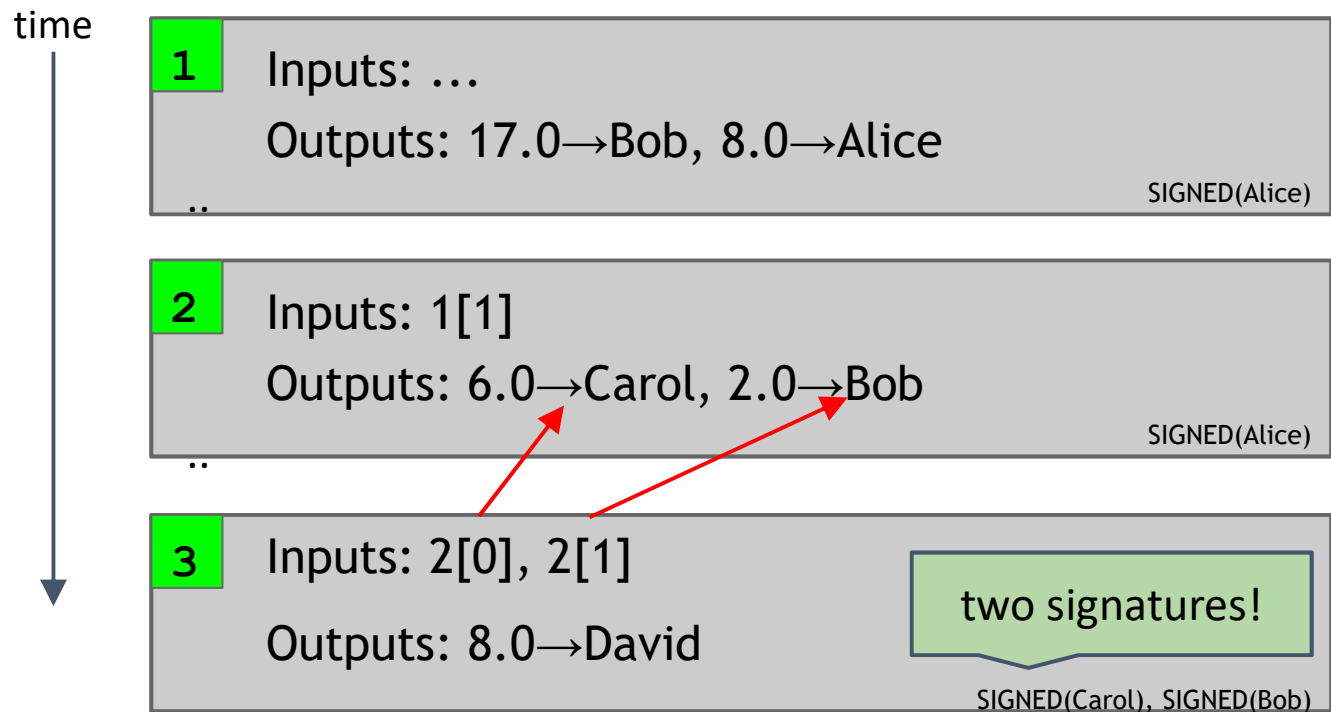
SIMPLIFICATION: only one transaction per block

# Merging value

- ***Consolidating funds.***

- As in Scroogecoin, since **transactions** can have *many inputs and many outputs*, **splitting and merging** value is easy.
- For example, suppose Bob received money in two different transactions—17 bitcoins in one, and 2 in another.
- Bob might want to have a single transaction output available for later, so that he can spend all 19 bitcoins he controls.
- That's easy—he creates a **transaction** using *the two inputs* and *one output*, with the output address being one that he owns.
- That lets Bob consolidate those two transactions.

# Joint payments



SIMPLIFICATION: only one transaction per block

# Joint payments

- ***Joint payments:***

- Similarly, joint payments are also easy to do.
- Suppose Carol and Bob both want to pay David.
- They can create a **transaction with two inputs and one output**, but with the ***two inputs owned by two different people***.
- And the only difference from the previous example is that since the two outputs from prior transactions that are being claimed here are from different addresses, the **transaction needs two separate signatures—one by Carol and one by Bob**.

# The real deal: a Bitcoin transaction

```
{
  "hash": "5a42590fbe0a90ee8e8747244d6c84f0db1a3a24e8f1b95b10c9e050990b8b6b",
  "ver": 1,
  "vin_sz": 2,
  "vout_sz": 1,
  "lock_time": 0,
  "size": 404,
  "in": [
    {
      "prev_out": {
        "hash": "3be4ac9728a0823cf5e2deb2e86fc0bd2aa503a91d307b42ba76117d79280260",
        "n": 0
      },
      "scriptSig": "30440..."
    },
    {
      "prev_out": {
        "hash": "7508e6ab259b4df0fd5147bab0c949d81473db4518f81afc5c3f52f91ff6b34e",
        "n": 0
      },
      "scriptSig": "3f3a4ce81...."
    }
  ],
  "out": [
    {
      "value": "10.12287097",
      "scriptPubKey": "OP_DUP OP_HASH160 69e02e18b5705a05dd6b28ed517716c894b3d42e OP_EQUALVERIFY OP_CHECKSIG"
    }
  ]
}
```

metadata

input(s)

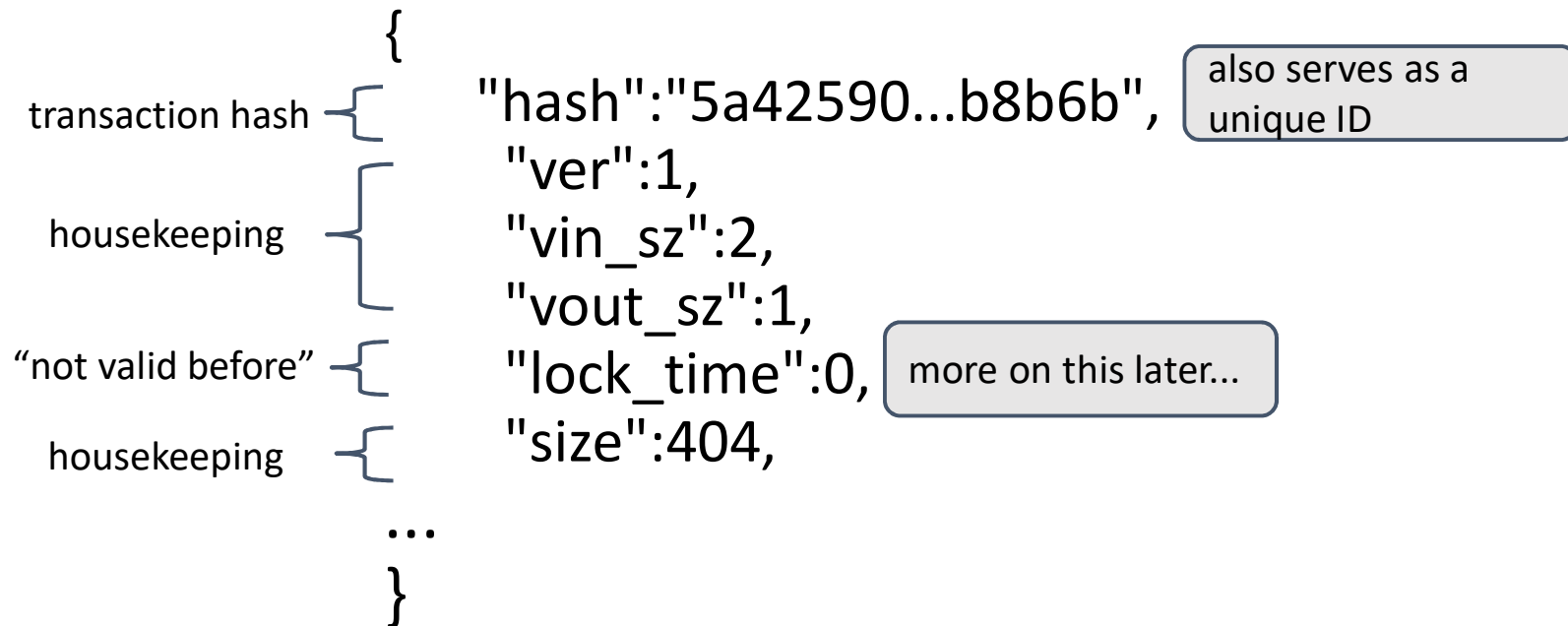
output(s)

# The real deal: a Bitcoin transaction

- ***Transaction syntax:***

- How transaction's are represented at a low level in Bitcoin.
- Every data structure that's sent on the network is a string of bits.
- What's shown in previous figure is low level, but this is further compiled down to a compact binary format that's not human readable.
- A **transaction** consists of three parts: **some metadata, a series of inputs, and a series of outputs**

# The real deal: transaction metadata



# The real deal: transaction metadata

- ***Metadata:***

- Some **housekeeping** information is **present—the size of the transaction, the number of inputs, and the number of outputs.**
- The **hash** of the **entire transaction** is supplied, which serves as a **unique ID for the transaction.**
- **This hash** allows us to use hash pointers to reference transactions.
- And a **lock\_time** field is supplied, which we'll come back to later.

## The real deal: transaction inputs

```
    "in":[  
      {  
        "prev_out":{  
          "hash":"3be4...80260",  
          "n":0  
        },  
        "scriptSig":"30440....3f3a4ce81"  
      },  
      ...  
    ],
```

previous transaction {

signature {

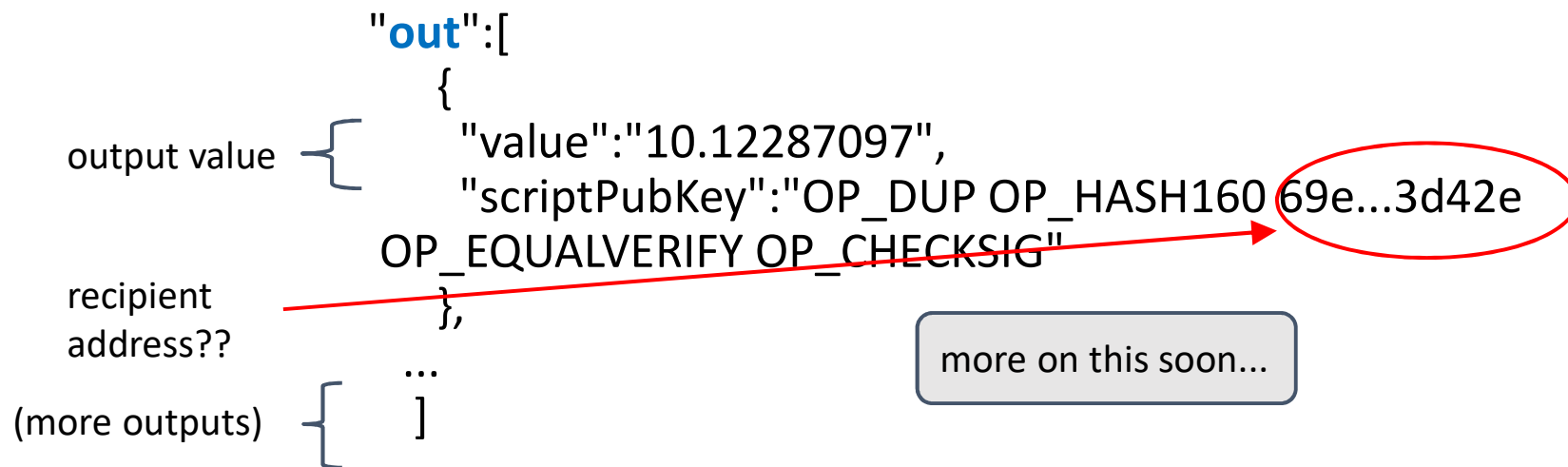
(more inputs) {

# The real deal: transaction inputs

- *Inputs:*

- The **transaction inputs form an array**, and each input has the same form.
- An **input specifies** a **previous transaction**, so it contains a hash of that transaction, which acts as a hash pointer to it.
- The **input also contains the index of the previous transaction's outputs** that are being claimed.
- And then there's a **signature**.
- Remember that **we have to sign to show** that **we actually have the ability to claim those previous transaction outputs**.

# The real deal: transaction outputs



**Sum of all output values less than or equal to sum of all input values!**  
If sum of all output values less than sum of all input values, then difference goes to miner as a transaction fee

# The real deal: transaction outputs

- ***Outputs:***

- The outputs are again an array.
- Each output has just **two fields**.
- They each have a value, and the *sum of all the output values has to be less than or equal to the sum of all the input values*.
- If the sum of the output values is less than the sum of the input values, *the difference is a transaction fee that goes to the miner who publishes this transaction*.

# Bitcoin scripts

# Bitcoin scripts

- Each **transaction output** doesn't just specify a **public key**. It **actually specifies a script**.
- The most common type of **transaction in Bitcoin is to redeem (correctly use) a previous transaction output by signing with the correct key**.
- To validate that a **transaction redeems a previous transaction output correctly, we combine the *new transaction's input script (scriptSig)* and the *earlier transaction's output script (scriptPubKey)***.
- An **address** is a **hash of a public key**

# Bitcoin scripts

- **The referenced transaction - Output Script (scriptPubKey):** This is part of the transaction where you originally received the Bitcoin. It includes the rules for spending this Bitcoin. Typically, it specifies the conditions that need to be met to redeem the Bitcoin, like providing a valid signature from the owner's private key.
- **The Input Script (scriptSig):** This is part of the new transaction where you're trying to use the Bitcoin. It includes a signature that proves you own the Bitcoin and meet the conditions specified in the scriptPubKey.

# Bitcoin scripts

- When a Bitcoin transaction is processed, **the network combines the scriptSig from the new transaction with the scriptPubKey from the previous transaction to verify that the conditions are met.**
- If the verification is successful, the Bitcoin is considered redeemed, and the transaction is confirmed.

# Bitcoin scripts

- So **merely specifying the address X** doesn't tell us what the **public key is, and doesn't give us a way to check the signature!**
- So instead the transaction output must say: **"this can be redeemed by a public key that hashes to X, along with a signature from the owner of that public key."**

## Output “addresses” are really *scripts*

- The following is an example **Pay-to-PubkeyHash script**, the most common type of output script in Bitcoin

```
OP_DUP  
OP_HASH160  
69e02e18...  
OP_EQUALVERIFY OP_CHECKSIG
```

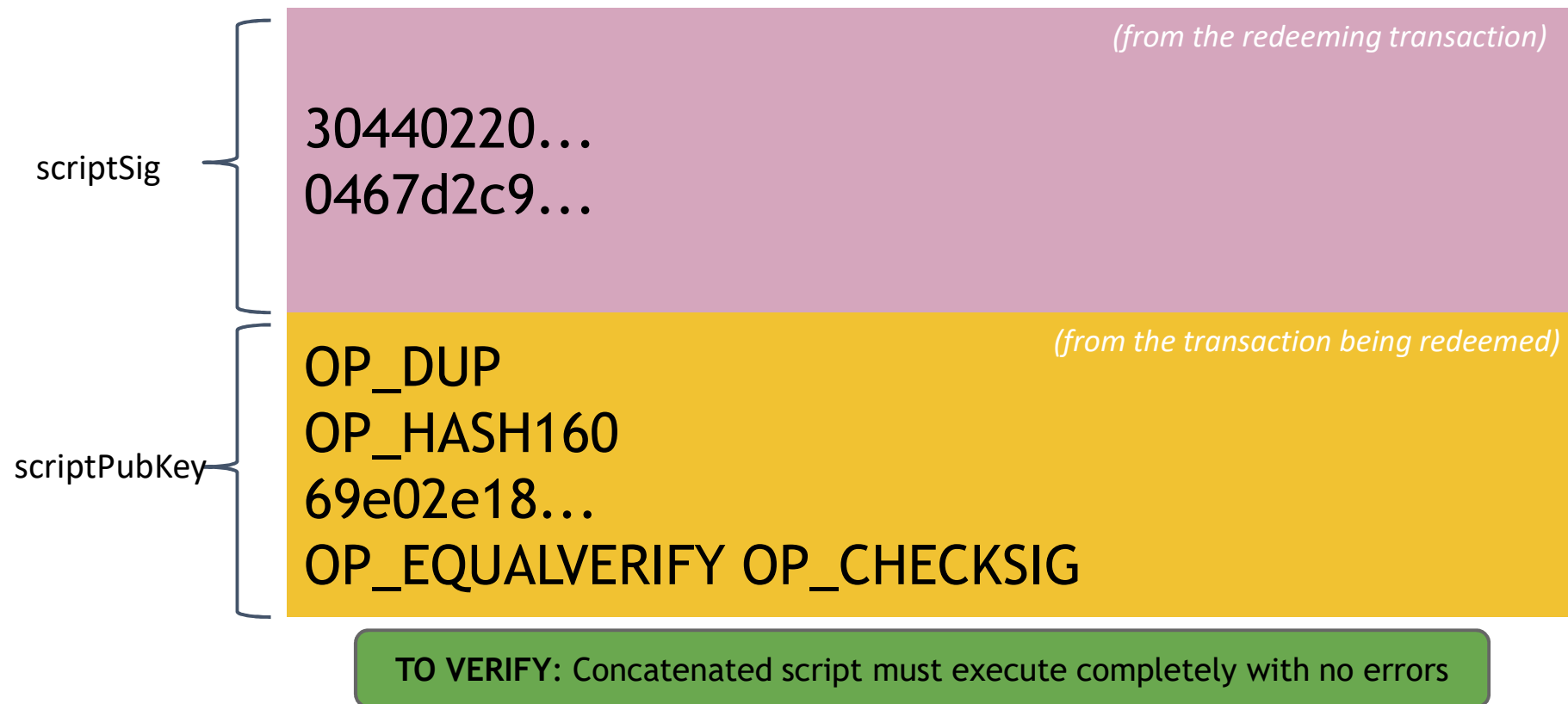
# Bitcoin scripts

- The secret is that the **inputs also contain scripts** instead of **signatures**.
- To validate that a **transaction redeems a previous transaction output correctly**, we **combine** the **new transaction's input script** and the **earlier transaction's output script**.
- We **simply concatenate** them, and the **resulting script must run successfully in order for the transaction to be valid**.
- These two scripts are called **scriptPubKey** and **scriptSig** because in the simplest case, the **output script just specifies a public key** (or **an address to which the public key hashes**), and the **input script** specifies a **signature with that public key**.

```
<sig>  
<pubKey>  
-----  
OP_DUP  
OP_HASH160  
<pubKeyHash?>  
OP_EQUALVERIFY  
OP_CHECKSIG
```

*Figure 3.5.* To check if a transaction correctly redeems an output, we create a combined script by appending the scriptPubKey of the referenced output transaction (bottom) to the scriptSig of the redeeming transaction (top). Notice that <pubKeyHash?> contains a '?'. We use this notation to indicate that we will later check to confirm that this is equal to the hash of the public key provided in the redeeming script.

Input “addresses” are *also* scripts



# Bitcoin scripting language (“Script”)

## Design goals

- Built for Bitcoin (inspired by Forth)
- Simple, compact
- Support for cryptography
- Stack-based (linear) –

every Instruction is executed exactly once

- Limits on time/memory - the number of instructions in the script gives us an upper bound on how long it might take to run and how much memory it could use.
- No looping
  - **Result:** Bitcoin script is not Turing Complete! i.e, cannot compute arbitrarily powerful functions.  
**Design:** **Miners** have to **run these scripts**, which are **submitted by arbitrary participants** in the **network**
  - **Advantage:** No infinite looping problem!

I am not  
impressed



image via Jessie St. Amand

## Bitcoin scripting language (“Script”)

- Bitcoin scripting language is very small
- 256 instructions (each represented by 1 byte)
  - **75 reserved, 15 disabled**
    - Basic arithmetic, basic logic (“if” → “then”), throwing errors, returning early, crypto instructions (hash computations, signature verifications), etc.
- Only two possible outcomes of a Bitcoin script
  - Executes successfully with no errors → transaction is valid OR
  - Error while execution → transaction invalid and should not be accepted in the block chain

# Common script instructions

| Name                    | Functions                                                                                                                      |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>OP_DUP</b>           | Duplicates top item on the stack                                                                                               |
| <b>OP_HASH160</b>       | Hashes twice: first using SHA-256, then using RIPEMD-160                                                                       |
| <b>OP_EQUALVERIFY</b>   | Returns true if inputs are equal, false (marks transaction invalid) otherwise                                                  |
| <b>OP_CHECKSIG</b>      | Checks that the input signature is valid using input public key for the hash of the current transaction                        |
| <b>OP_CHECKMULTISIG</b> | Checks that $t$ signatures on the transaction are valid from $t$ ( <i>out of <math>n</math></i> ) of the specified public keys |

# RIPEMD

- “RIPEMD” stands for “RIPE Message Digest,” where “RIPE” stands for “RACE Integrity Primitives Evaluation” and where “RACE” stands for “Research and Development in Advanced Communications Technologies in Europe”—a nice example of a recursive abbreviation.

## OP\_CHECKMULTISIG

- Built-in support for joint signatures
- Specify  $n$  public keys
- Specify  $t$  - threshold
- Verification requires  $t$  signatures are valid



**BUG ALERT:** Extra data value popped from the stack and ignored

- One has to deal with this bug by putting an extra dummy variable onto the stack.

# Bitcoin script execution

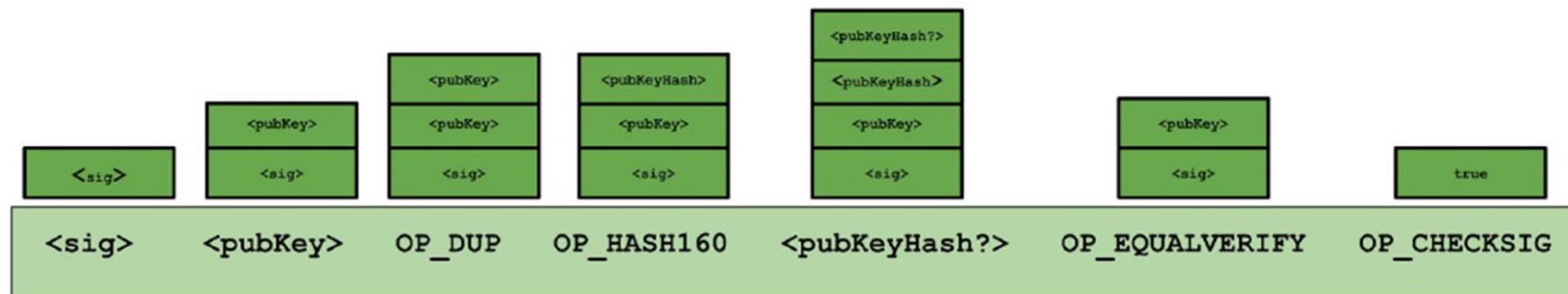
- **Stack** that **we can push data to** and **pop data from**.
- No need of any other memory or variables.
- There are two types of instructions: **data instructions and opcodes**.
- When a **data instruction appears in a script**, that **data is simply pushed onto the top of the stack**.
- **Opcodes**, on the other hand, **perform some function**, often taking as **input data that is on top of the stack**.

```
<sig>  
<pubKey>  
-----  
OP_DUP  
OP_HASH160  
<pubKeyHash?>  
OP_EQUALVERIFY  
OP_CHECKSIG
```

*Figure 3.5.* To check if a transaction correctly redeems an output, we create a combined script by appending the scriptPubKey of the referenced output transaction (bottom) to the scriptSig of the redeeming transaction (top). Notice that <pubKeyHash?> contains a '?'. We use this notation to indicate that we will later check to confirm that this is equal to the hash of the public key provided in the redeeming script.

# Bitcoin script execution

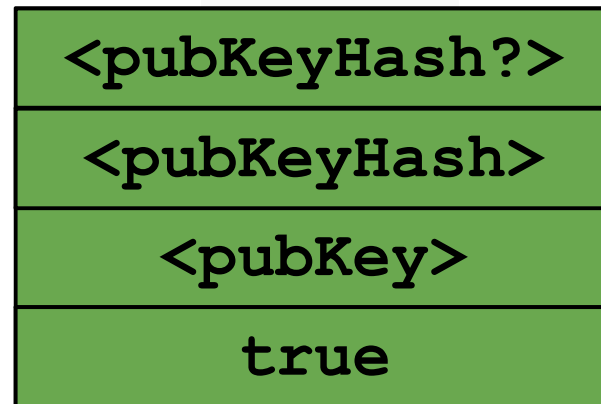
- The **first two instructions in this script are data instructions** — the **signature** and the **public key used to verify that signature** — specified in the **scriptSig** component of a **transaction input** in the **redeeming transaction**.
- The rest of the script was specified in the **scriptPubKey** component of a **transaction output** in the **referenced transaction**.



**Figure 3.7 Execution of a Bitcoin script.** On the bottom, we show the instruction in the script. Data instructions are denoted with surrounding angle brackets, whereas opcodes begin with “OP\_”. On the top, we show the stack just after that instruction has been executed.

- There is **the hash of the public key**, as **specified by the sender**, and the **hash of the public key** that was **used by the recipient** when **trying to claim the coins**.
- **EQUALVERIFY** command, which checks that the two values at the top of the stack are **equal**. If they aren't, **an error will be thrown, and the script will stop executing**.
- **“CHECKSIG”** instruction pops the two values, the public key and signature, off the stack, and verifies that is a **valid signature for the entire transaction using that public key**.
- Provided there **weren't any errors**, the output of this script will **simply be true indicating that the transaction is valid**.

# Bitcoin script execution example



Below the stack, a horizontal light green box contains the script: `<sig> <pubKey> OP_DUP OP_HASH160 <pubKeyHash?> OP_EQUALVERIFY OP_CHECKSIG`. Above this box, seven black downward-pointing arrows indicate the stack state at each step: the first four arrows point to the `<pubKey>`, `<pubKeyHash>`, `<pubKeyHash?>`, and `true` elements respectively; the fifth arrow points to the space between `OP_HASH160` and `<pubKeyHash?>`; the sixth arrow points to the space between `OP_EQUALVERIFY` and `OP_CHECKSIG`; and the seventh arrow points to the space after the final operation.

`<sig> <pubKey> OP_DUP OP_HASH160 <pubKeyHash?> OP_EQUALVERIFY OP_CHECKSIG`

# Bitcoin scripts in practice (as of 2014)

- **Scripts** let us specify **arbitrary conditions** that **must be satisfied to spend coins**.
- Most nodes whitelist known scripts - they **refuse to accept scripts that are not on the list**.
- 99.9% are simple signature checks - this script **just specifies one public key** and **requires a signature** for that **public key** in order to **spend the coins**.
- ~0.01% are MULTISIG
- ~0.01% are **Pay-to-Script-Hash**
- Remainder are errors, proof-of-burn



More on this soon

# Proof-of-burn

- A **proof-of-burn** is a **script** that can never be **redeemed**.
- **Sending coins to a proof-of-burn script establishes** that they have been **destroyed** since **there's no possible way for them to be spent**.
- One **use of proof-of-burn is to bootstrap an alternative to Bitcoin by forcing people to destroy Bitcoin in order to gain coins in the new system**.
- **Bootstrapping in the context of cryptocurrencies refers to the method of *launching and growing a new cryptocurrency without relying on external funding or resources***.

# Proof-of-burn

- Proof-of-burn is quite simple to implement: the **OP\_RETURN opcode throws an error if it's ever reached**.
- No matter what values you put before **OP\_RETURN**, that instruction will get executed eventually, in which case **this script will return false**.
- Because the **error is thrown**, the **data in the script** that **comes after OP\_RETURN will not be processed**.
- **An opportunity for people to put arbitrary data in a script, and hence into the block chain.**
- **OP\_RETURN <arbitrary\_data>**

# Proof-of-burn

- If, for some reason, **you want to write your name**, or **if you want to timestamp** and **prove that you knew some data at a specific time**, you can **create a very low value Bitcoin transaction**.
- You can **destroy a very small amount of currency**, but **you get to write whatever you want into the block chain**, which **should be kept around forever**.

## Proof-of-Burn

---

this script can never be redeemed 😞

```
OP_RETURN  
<arbitrary data>
```



Uses for Proof-of-Burn:

- Destroy coins and transfer them to alternative currency
  - Add arbitrary data to block chain
-

# Proof-of-burn

nothing's going to redeem that 😞

OP\_RETURN  
<arbitrary data>

# Pay-to-script-hash

- Bitcoin scripts works is that the sender of coins has to specify the script exactly.
- For example, you're a **consumer shopping online**, and you're about to order something.
- And you say, "Alright, I'm ready to pay.
- Tell me the address to which I should send my coins."
- Now, say that the **company** that you're ordering from is using **MULTISIG addresses**.
- Then, since the one spending the coins has to specify this, the retailer will have to come back and say, "Oh, well, we're doing something fancy now.
- We're using MULTISIG. We're going to ask you to send the coins to some complicated script."
- You might say, "I don't know how to do that. That's too complicated. As a consumer, I just want to send to a simple address."

# Pay-to-script-hash

- Bitcoin has a clever solution to this problem, and it applies to not just multi-sig addresses but to any complicated condition governing when coins can be spent.
- **Instead of telling the sender “send your coins to the hash of this public key”, the receiver can instead tell the sender “send your coins to the hash of this *script*”.**
- **Impose the condition that to redeem those coins, it is necessary to reveal the script that has the given hash, and further, provide data that will make the script evaluate to true.**
- The sender achieves this by using the Pay-to-script-hash (P2SH) transaction type, which has the above semantics.
- P2SH wasn't part of Bitcoin's initial design specification.

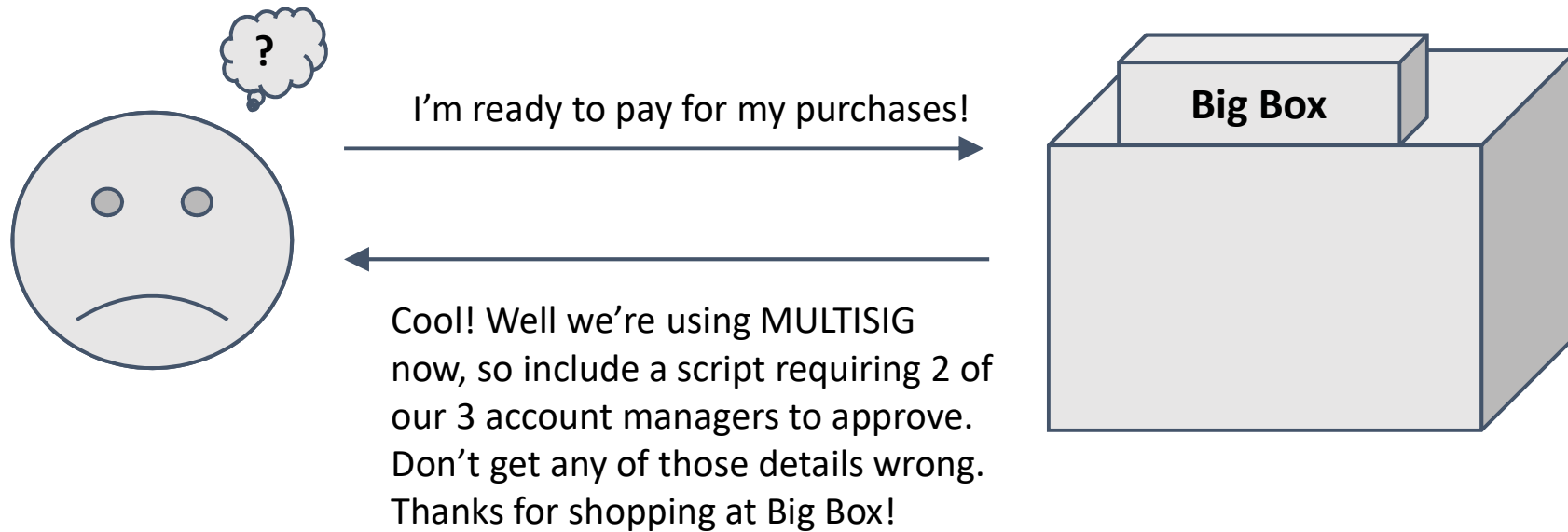
# Pay-to-script-hash

- P2SH script simply hashes the top value on the stack, checks if it matches the provided hash value.
- Then executes a special second step of validation: that top data value from the stack is reinterpreted as a sequence of instructions, and executed a second time as a script, with the rest of the stack as input.
- Getting support for P2SH was quite complicated since it wasn't part of Bitcoin's initial design specification and added later.
- The recipient can just specify a hash that the sender sends money to.

# Pay-to-script-hash (P2SH)

- The P2SH script simply hashes the top value on the stack, checks whether it matches the provided hash value,
- And then executes a special second step of validation: that top data value from the stack is reinterpreted as a sequence of instructions and is executed a second time as a script, with the rest of the stack as input.
- P2SH **removes the need for a complex response from the sender**, because the *recipient can just specify a hash that the sender sends money to.*
- *With P2SH outputs, the output scripts are now much smaller, as they only specify a hash.*
- *All of the complexity is pushed to the input scripts.*
- **P2SH also has a nice efficiency gain.**

# Should senders specify scripts?



# Pay-to-script-hash

- In our example above, **Alice** need not worry that **Bob is using multisig**; **she** just **sends** to **Bob's P2SH address**, and it is **Bob's responsibility to specify the fancy script** when **he wants to redeem the coins**.

## Pay-to-Script-Hash (P2SH) Workflow

Bob

- creates a redeem script with whatever script he wants
- hashes the redeem script
- sends redeem script hash to Alice.

Alice

- creates a P2SH-style output containing Bob's redeem script hash.

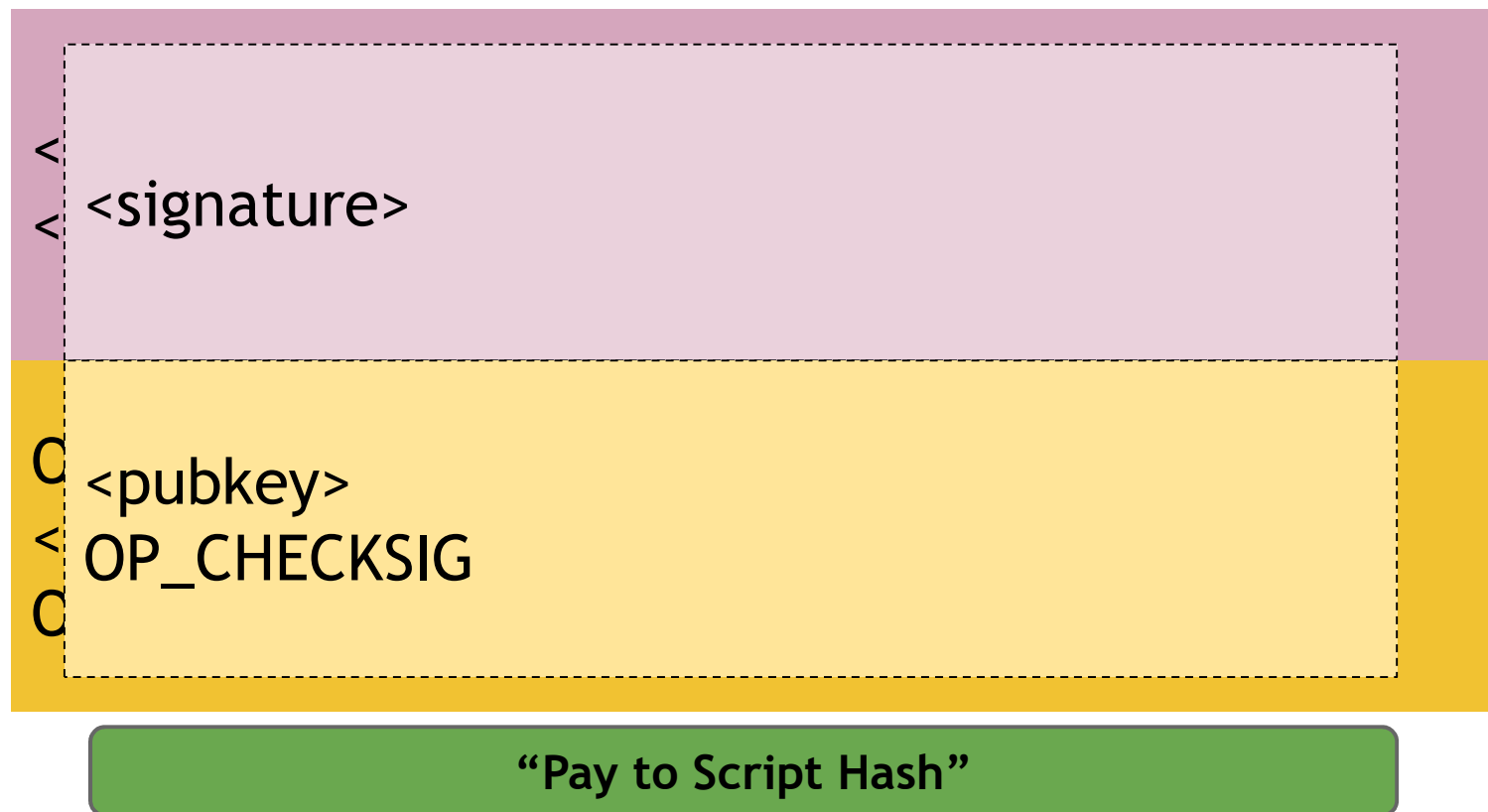
When Bob wants to spend the output

- provides his signature along with the redeem script in the signature script.

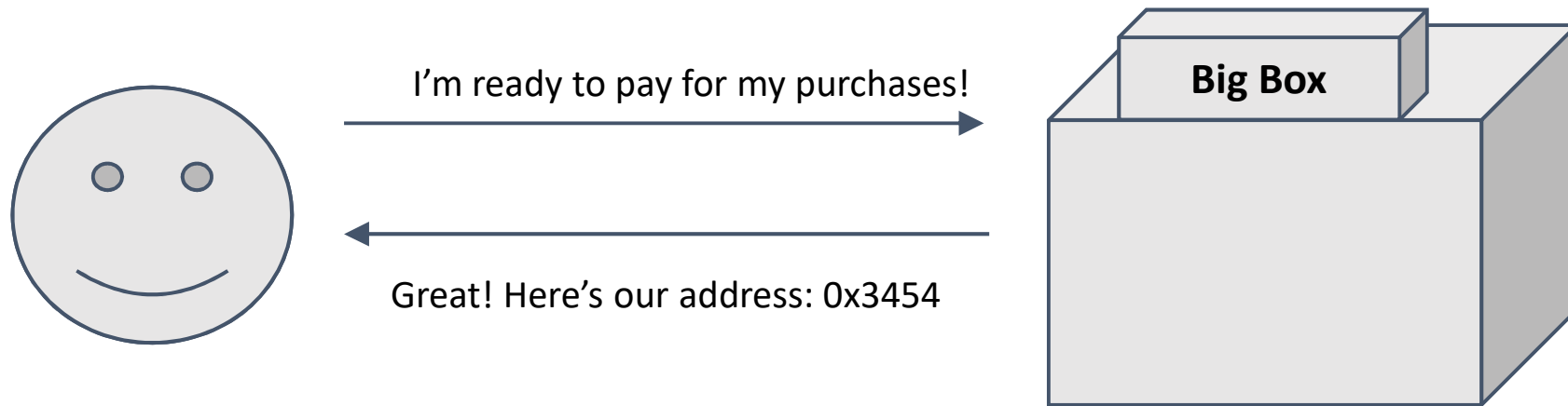
P2P network then

- ensures the redeem script hashes to the same value as the script hash Alice put in her output;
- it then processes the redeem script exactly as it would if it were the primary pubkey script,
- letting Bob spend the output if the redeem script does not return false.

Idea: use the hash of redemption script



# Pay to script hash



# Applications of Bitcoin scripts

# Escrow transactions

- Say Alice and Bob want to do business with each other — Alice wants to pay Bob in Bitcoin for Bob to send some physical goods to Alice.
- The problem though is that Alice doesn't want to pay until after she's received the goods, but Bob doesn't want to send the goods until after he has been paid.
- What can we do about that? A nice solution in Bitcoin that's been used in practice is **to introduce a third party and do an escrow transaction.**

# Escrow transactions

- **Escrow transactions** can be implemented quite simply using **MULTISIG**.
- **Alice** doesn't send the **money directly to Bob**, but **instead creates a MULTISIG transaction** that requires **two of three people to sign** in order to redeem the coins.
- And those **three people** are going to be **Alice, Bob, and some third party arbitrator, Judy**, who will **come into play in case there's any dispute**.

# Escrow transactions

- So **Alice creates a 2-of-3 MULTISIG transaction** that sends some coins she owns and specifies that they can be spent if any two of **Alice, Bob, and Judy sign**.
- **This transaction is included in the block chain, and at this point, these coins are held in escrow between Alice, Bob, and Judy, such that any two of them can specify where the coins should go.**
- **At this point, Bob is convinced that it's safe to send the goods over to Alice, so he'll mail them or deliver them physically.**

# Escrow transactions

- Now in **the normal case, Alice and Bob are both honest.**
- **Bob will send over the goods** that Alice is expecting, and when **Alice receives the goods, Alice and Bob both sign a transaction redeeming the funds from escrow, and sending them to Bob.**
- Notice that in this case where both Alice and Bob are honest, Judy never had to get involved at all.
- There was no dispute, and Alice's and Bob's signatures met the 2-of-3 requirement of the MULTISIG transaction.

# Escrow transactions

- In the **normal case**, it requires just **one extra transaction on the block chain**.
- **But what would have happened if Bob didn't actually send the goods or they got lost in the mail?**
- **Or perhaps the goods were different than what Alice ordered?**
- **Alice now doesn't want to pay Bob because she thinks that she got cheated, and she wants to get her money back.**
- **So Alice is definitely not going to sign a transaction that releases the money to Bob.**

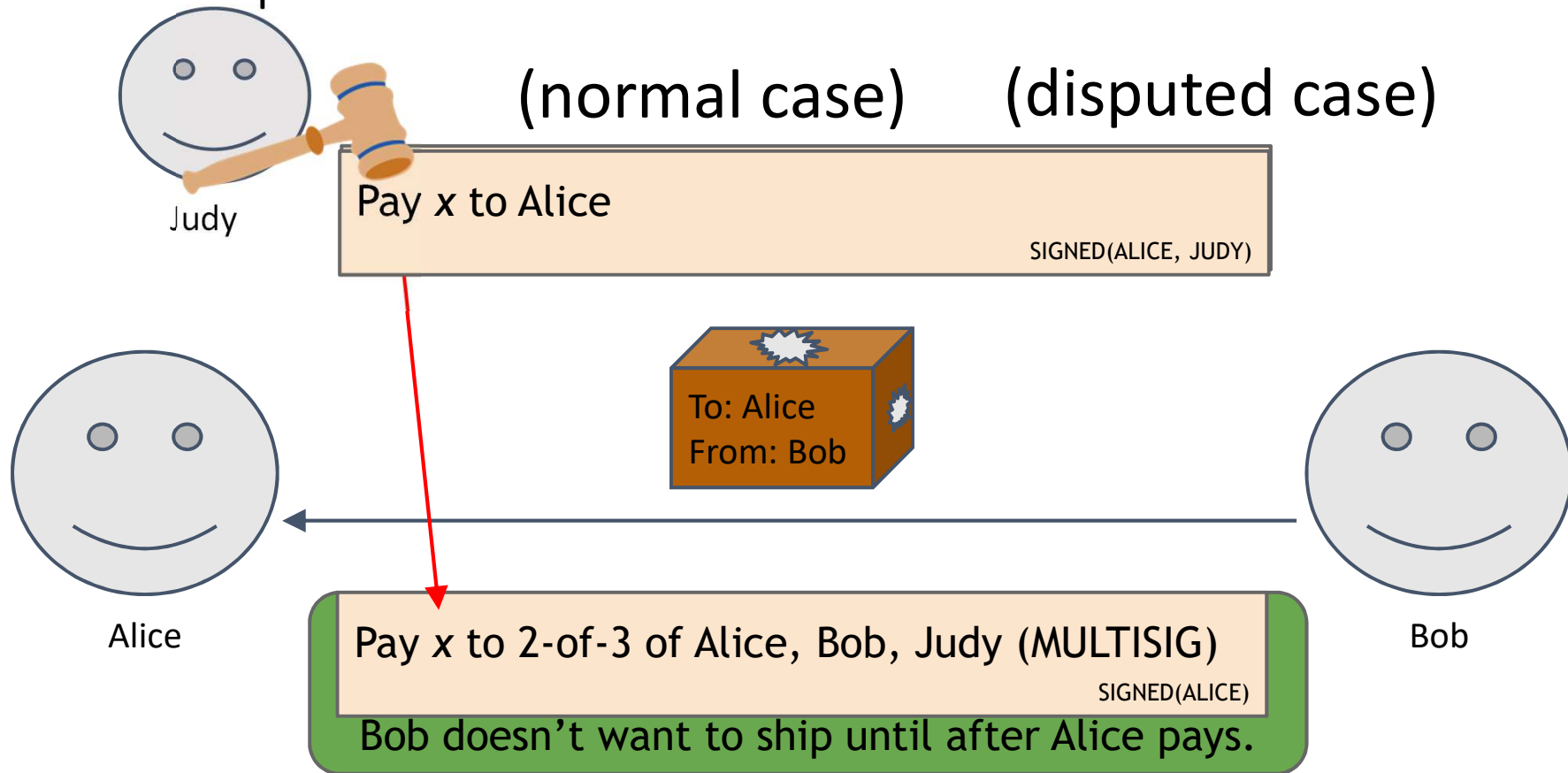
# Escrow transactions

- But **Bob also may deny any wrongdoing** and **refuse to sign a transaction** that **releases the money back to Alice**.
- This is where **Judy needs to get involved**.
- **Judy's going to have to decide which of these two people deserves the money**.
- If **Judy decides that Bob cheated**, **Judy will be willing to sign a transaction along with Alice, sending the money from escrow back to Alice**.

# Escrow transactions

- Alice's and Judy's signatures meet the 2-of-3 requirement of the MULTISIG transaction, and Alice will get her money back.
- If Judy thinks that Alice is at fault here, and Alice is simply refusing to pay when she should, Judy can sign a transaction along with Bob, sending the money to Bob.
- So Judy decides between the two possible outcomes.
- But the nice thing is that Judy won't have to be involved unless there's a dispute.

## Example 1: Escrow transactions



# Green addresses

- Say Alice wants to pay Bob, and Bob's offline.
- Since he's offline, Bob can't go and look at the block chain to see if a transaction that Alice is sending is actually there.
- It's also possible that Bob is online, but doesn't have the time to go and look at the block chain and wait for the transactions to be confirmed.
- Remember that normally we want a **transaction to be in the block chain and be confirmed by six blocks**, which **takes up to an hour**, before we **trust that it's really in the block chain**.

# Green addresses

- To solve this problem of being able to send money using Bitcoin without the recipient being able to access the block chain, we have to introduce another third party, which we'll call the bank (in practice it could be an exchange or any other financial intermediary).
- Alice is going to talk to her bank, and say, "Hey, it's me, Alice. I'm your loyal customer. Here's my card or my identification."
- And I'd really like to pay Bob here, could you help me out?"
- And the bank will say, "Sure. I'm going to deduct some money out of your account."
- And draw up a transaction from one of my green addresses over to Bob

# Green addresses

- So notice that this money is coming directly from the bank to Bob. Some of the money, of course, might be in a change address going back to the bank.
- But essentially, the bank is paying Bob here from **a bank-controlled address**, which we call a green address.
- Moreover, the **bank guarantees** that it **will not double-spend** this **money**.
- So as soon as **Bob sees that this transaction is signed by the bank**, if **he trusts the bank's guarantee not to double-spend the money**, he can accept that that **money will** eventually be **his when it's confirmed in the block chain**.
- Notice that this is **not a Bitcoin-enforced guarantee**.

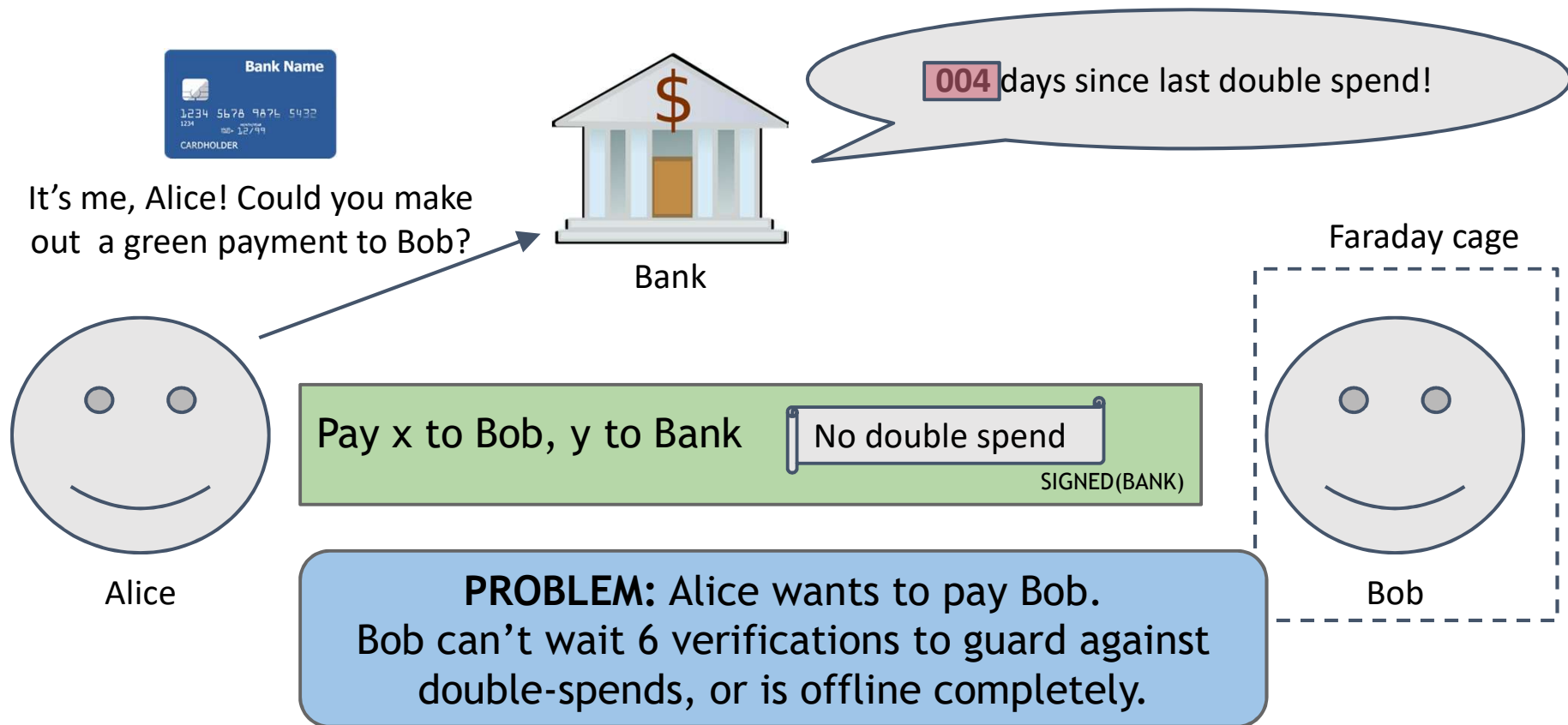
# Green addresses

- This is a real-world guarantee, and in order for this system to work, Bob has to trust that the bank, in the real world, cares about their reputation and won't double-spend for that reason.
- And the bank will be able to say, "You can look at my history. I've been using this green address for a long time, and I've never double spent.
- Therefore I'm very unlikely to do so in the future." Thus Bob no longer has to trust Alice, whom he may know nothing about.
- Instead, he places his trust in the bank that they will not double-spend the money that they sent him.

# Green addresses

- if the bank ever does double-spend, people will stop trusting its green address(es).
- In fact, the **two most prominent online services** that implemented **green addresses** were **Instawallet and Mt. Gox**, and both ended up collapsing.
- **Today green addresses aren't used very much.**

## Example 2: Green addresses



# Efficient micro-payments

- Alice is a customer who wants to continually pay Bob small amounts of money for some service that Bob provides.
- For example, Bob may be Alice's wireless service provider, and requires her to pay a small fee for every minute that she talks on her phone.
- Creating a Bitcoin transaction for every minute that Alice speaks on the phone won't work.
- That will create too many transactions, and the transaction fees add up. If the value of each one of these transactions is on the order of what the transaction fees are, Alice is going to be paying quite a high cost to do this.

# Efficient micro-payments

- What we'd like is to be able to combine all these small payments into one big payment at the end.
- It turns out that there's a neat way to do this.
- We start with a MULTISIG transaction that pays the maximum amount Alice would ever need to spend to an output requiring both Alice and Bob to sign to release the coins.
- Now, after the first minute that Alice has used the service, or the first time Alice needs to make a micropayment, she signs a transaction spending those coins that were sent to the MULTISIG address, sending one unit of payment to Bob and returning the rest to Alice.
- After the next minute of using the service, Alice signs another transaction, this time paying two units to Bob and sending the rest to herself.

# Efficient micro-payments

- Notice these are signed only by Alice, and haven't been signed by Bob yet, nor are they being published to the block chain.
- Alice will keep sending these transactions to Bob every minute that she uses the service.
- Eventually, Alice will finish using the service, and tells Bob, "I'm done, please cut off my service." At this point Alice will stop signing additional transactions.
- Upon hearing this, Bob will say "Great.
- I'll disconnect your service, and I'll take that last transaction that you sent me, sign it, and publish that to the block chain."

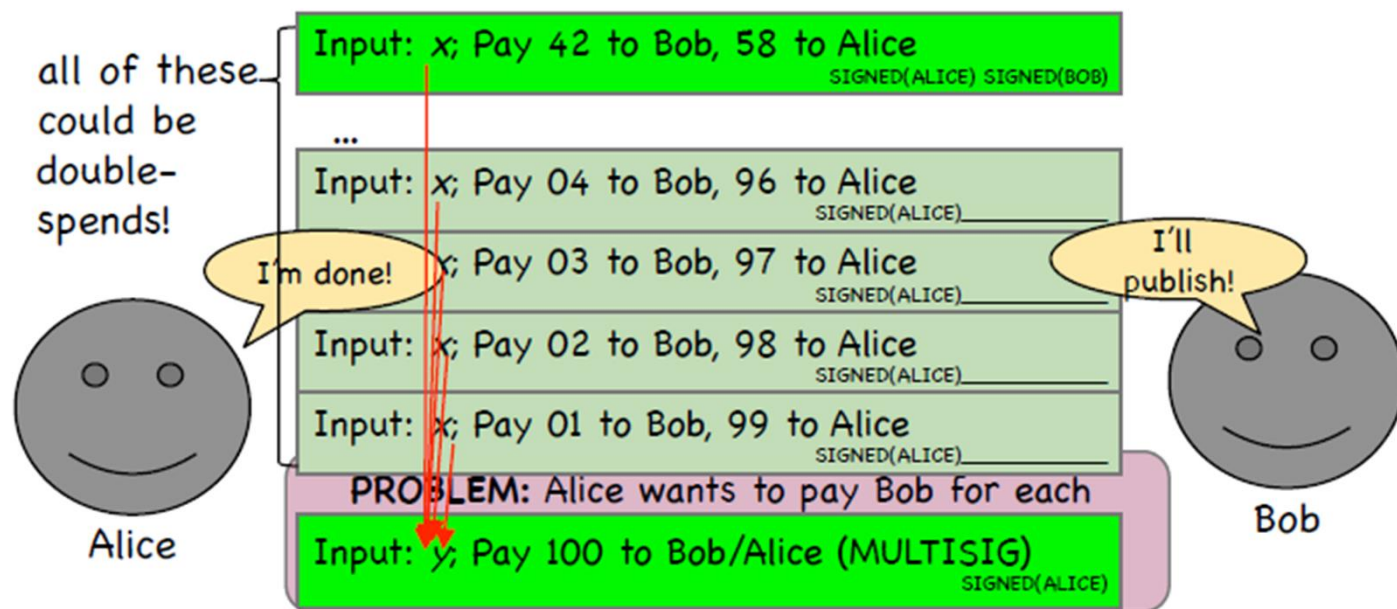
# Efficient micro-payments

- Technically all of these transactions are double-spends.
- So unlike the case with green addresses where we were specifically trying to avoid double-spends, with a strong guarantee, with this micro-payment protocol, we're actually generating a huge amount of potential double-spends.
- In practice, however, if both parties are operating normally, Bob will never sign any transaction but the last one, in which case the block chain won't actually see any attempt at a double-spend.

# Efficient micro-payments

- What if Bob **never signs the last transaction**?
- He may just say, “I’m happy to let the coins sit there in escrow forever,” in which case, maybe the coins won’t move, but Alice will lose the full value that she paid at the beginning.

## Example 3: Efficient Micro-Payments



## Example 3: Efficient Micro-Payments

---

What if Bob never signs??

Input:  $x$ ; Pay 42 to Bob, 58 to Alice

SIGNED(ALICE)\_\_\_\_\_

Alice demands a **timed refund transaction** before starting

Input:  $x$ ; Pay 100 to Alice, LOCK until time  $t$

SIGNED(ALICE) SIGNED(BOB)



Alice



Bob

Input:  $y$ ; Pay 100 to Bob/Alice (MULTISIG)

SIGNED(ALICE)

# Lock time

- To avoid this problem, before the micro-payment protocol can even start, Alice and Bob will both sign a transaction which refunds all of Alice's money back to her, but the refund is "locked" until some time in the future.
- So after Alice signs, but before she broadcasts, the first MULTISIG transaction that puts her funds into escrow, she'll want to get this refund transaction from Bob and hold on to it.
- That guarantees that if she makes it to time  $t$  and Bob hasn't signed any of the small transactions that Alice has sent, Alice can publish this transaction which refunds all of the money directly to her.

```
lock_time
{
  "hash":"5a42590...b8b6b",
  "ver":1,
  "vin_sz":2,
  "vout_sz":1,
  "lock_time":315415,
  "size":404,
  ...
}
```

Block index or real-world timestamp before  
which this transaction can't be published

- The way it works is that if you specify **any value other than zero for the lock time**, it **tells miners not to publish the transaction until the specified lock time**.

# Lock time

- The transaction will be invalid before a specific block number.
- So this is a way of preparing a transaction that can only be spent at some future time— provided the coin it's trying to spend hasn't already been spent by then in some other transaction.
- It works quite nicely in the micropayment protocol as a **safety valve to reassure Alice that if Bob never signs, eventually she'll be able to get her money back.**

## More advanced scripts

- Multiplayer lotteries
- Hash pre-image challenges
- Coin-swapping protocols
  - Don't miss the lecture on anonymity!

**“Smart contracts”**

# Smart contracts

- These are contracts for which we have **some degree of technical enforcement in Bitcoin**, whereas traditionally they are enforced through laws or courts of arbitration.
- It's a really cool feature of Bitcoin that **we can use scripts, miners, and transaction validation** to **realize the escrow protocol or the micro-payment protocol without needing a centralized authority**.
- There are many types of smart contracts which people would like to be able to enforce but which aren't supported by the Bitcoin scripting language today (2014).
- Nobody has come up with a creative way to implement them (2014).

Bitcoin blocks

## Bitcoin Blocks

---

**Q:** Why **bundle** transactions together?

1. Requiring consensus for each transaction separately would **reduce transaction acceptance rate**.
2. Hash-chain of blocks is much **shorter**.
3. Faster to **verify** history.

# Bitcoin blocks

Why bundle transactions together?

- Single unit of work for miners
- Limit length of hash-chain of blocks
  - Faster to verify history

# Bitcoin blocks

- A hash chain of blocks is much shorter than a hash chain of transactions would be, since a large number of transactions can be put into each block.
- This will make it much more efficient to verify the block chain data structure.

# Bitcoin blocks

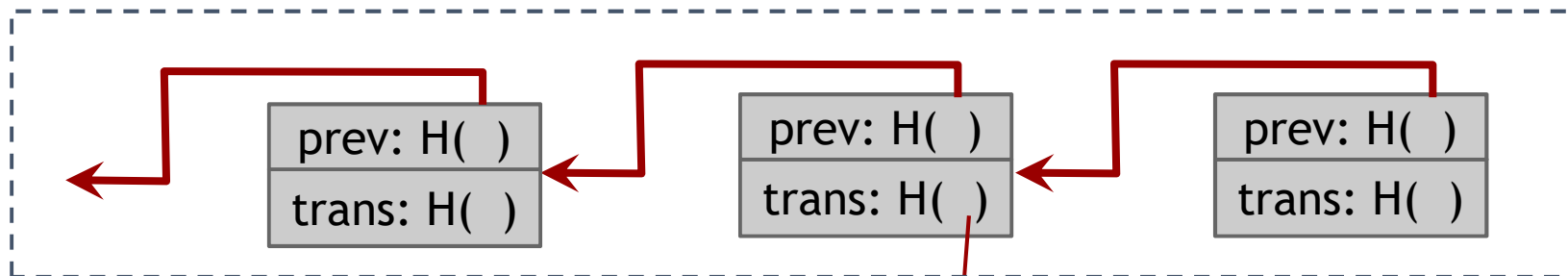
- The block chain is a clever combination of two different hash-based data structures.
- **The first is a hash chain of blocks.**
- **Each block has a block header, a hash pointer to some transaction data, and a hash pointer to the previous block in the sequence.**
- The **second data structure is a per-block tree** of all of **the transactions that are included in that block.**
- **This is a Merkle tree and allows us to have a digest of all the transactions in the block in an efficient way.**

# Bitcoin blocks

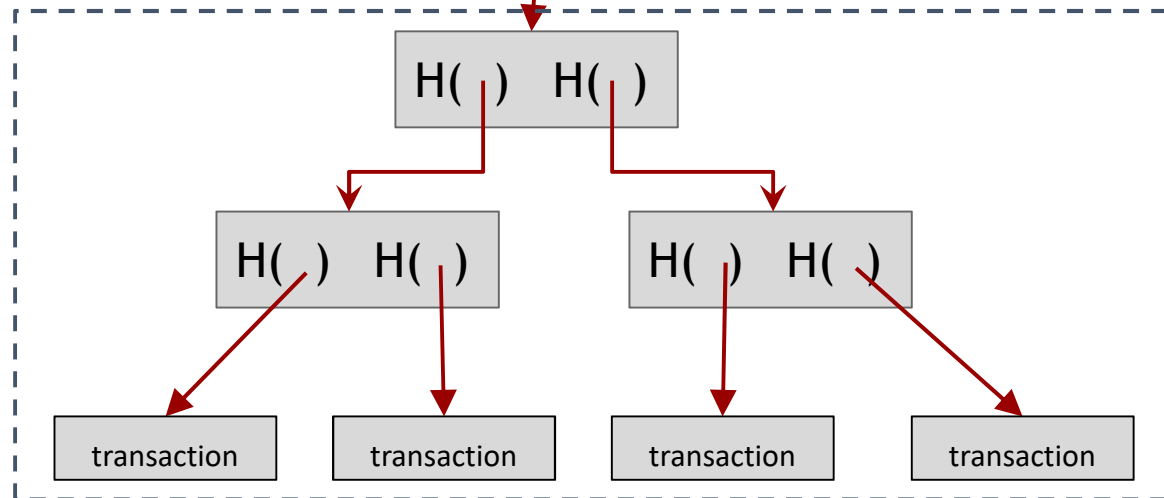
- To **prove that a transaction is included in a specific block, we can provide a path through the tree whose length is logarithmic in the number of transactions in the block.**
- To recap, a block consists of header data followed by a list of transactions arranged in a tree structure.

# Bitcoin block structure

Hash chain of blocks



Hash tree (Merkle tree) of transactions in each block



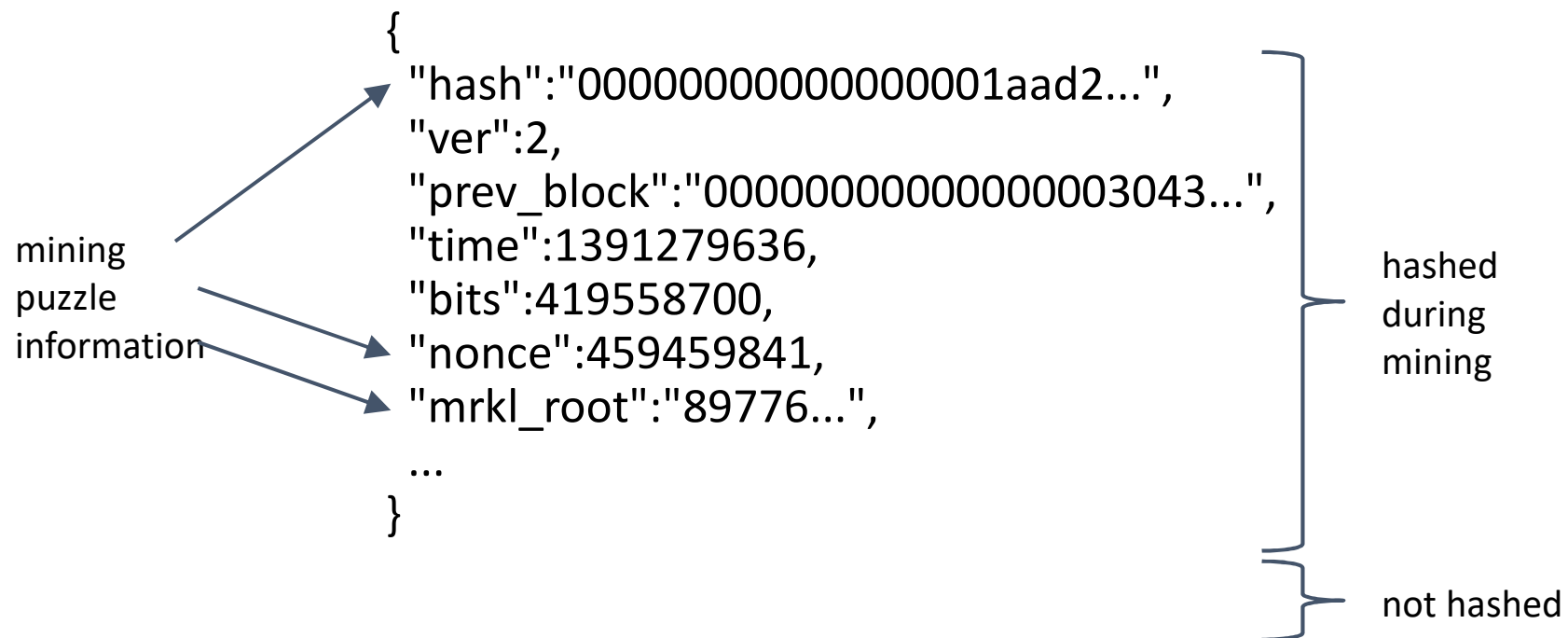
# The real deal: a Bitcoin block

- Header mostly contains information related to the mining puzzle.
- The hash of the block header has to start with a large number of zeros for the block to be valid.
- The header also contains a “nonce” that miners can change, a time stamp, and “bits”, which is an indication of how difficult this block was to find.
- **The header is the only thing that’s hashed during mining.**
- So to **verify a chain of blocks, all we need to do is look at the headers.**
- The **only transaction data** that’s **included in the header is the root of the transaction tree — the “mrkl\_root” field.**

# The real deal: a Bitcoin block

block header {  
"hash": "00000000000000001aad2...",  
"ver": 2,  
"prev\_block": "00000000000000003043...",  
"time": 1391279636,  
"bits": 419558700,  
"nonce": 459459841,  
"mrkl\_root": "89776...",  
"n\_tx": 354,  
"size": 181520,  
"tx": [  
...  
],  
"mrkl\_tree": [  
"6bd5eb25...",  
...  
"89776cdb..."  
]  
}  
  
transaction data

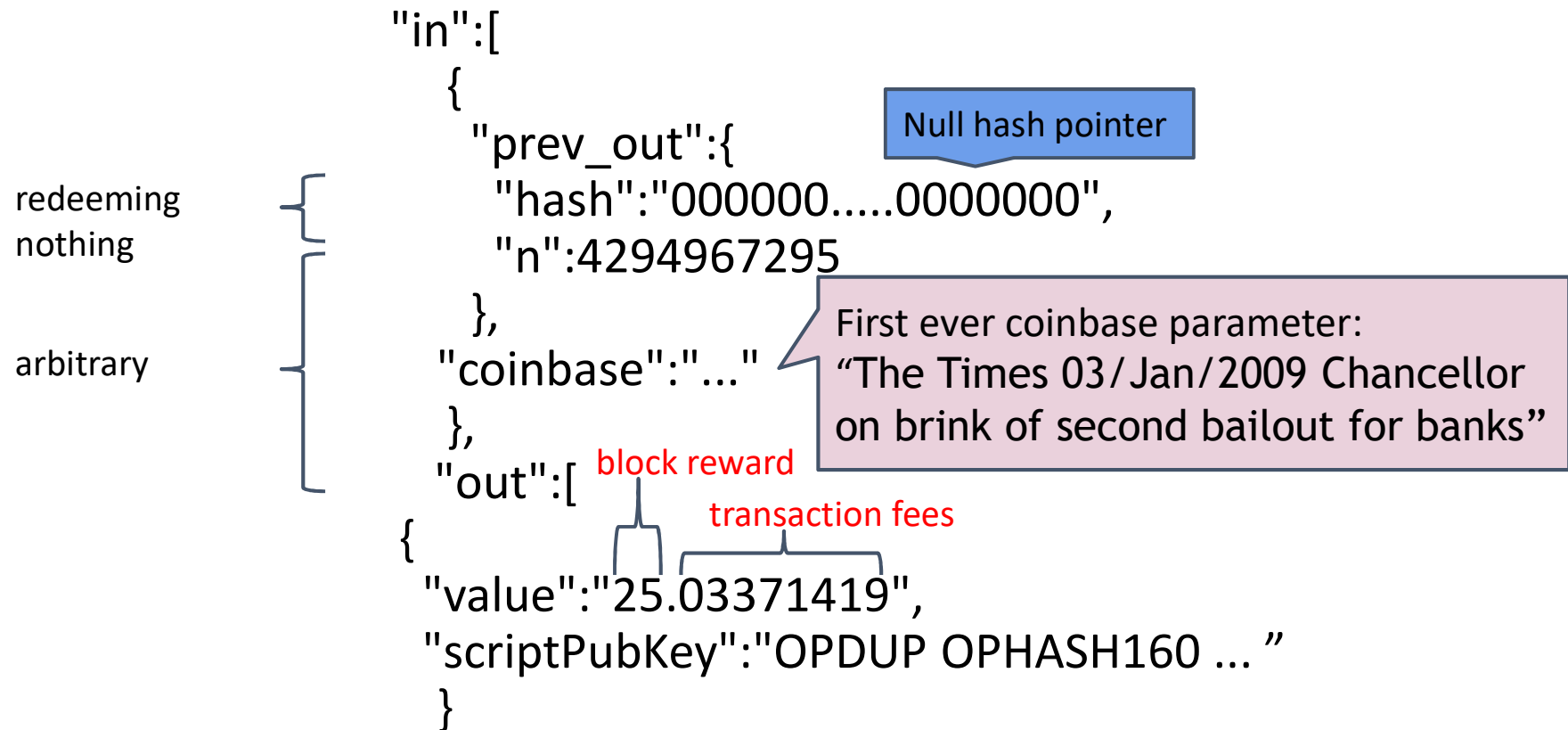
# The real deal: a Bitcoin block header



# The real deal: coinbase transaction

- It mostly looks like a normal transaction but with several differences:
  - (1) it always has a single input and a single output,
  - (2) the input doesn't redeem a previous output and thus contains a null hash pointer, since it is minting new bitcoins and not spending existing coins,
  - (3) the value of the output is currently a little over 25 Bitcoins.
    - The output value is the miner's revenue from the block. It consists of two components: a flat mining reward, which is set by the system and which halves every 210,000 blocks (about 4 years), and the transaction fees collected from every transaction included in the block.
  - (4) There is a special "coinbase" parameter, which is completely arbitrary — miners can put whatever they want in there.

# The real deal: coinbase transaction



# See for yourself!

## Transaction View information about a bitcoin transaction

151b750d1f13e76d84e82b34b12688811b23a8e3119a1cba4b4810f9b0ef408d

1KryFUt9tXHvaoCYTNPbqpWPJKQ717YmL5



1KvdrQ3oGqMAiDTMEYCcdDSnVaGNW2YZh  
1KryFUt9tXHvaoCYTNPbqpWPJKQ717YmL5

1.0194 BTC

3.458 BTC

9 Confirmations

4.4774 BTC

### Summary

|                                                                                                   |                                                         |
|---------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Size                                                                                              | 257 (bytes)                                             |
| Received Time                                                                                     | 2014-08-05 01:55:25                                     |
| Included In Blocks                                                                                | <a href="#">314018</a> (2014-08-05 02:00:40 +5 minutes) |
| Confirmations                                                                                     | 9 Confirmations                                         |
| Relayed by IP  | <a href="#">Blockchain.info</a>                         |
| Visualize                                                                                         | <a href="#">View Tree Chart</a>                         |

### Inputs and Outputs

|                          |                                             |
|--------------------------|---------------------------------------------|
| Total Input              | 4.4775 BTC                                  |
| Total Output             | 4.4774 BTC                                  |
| Fees                     | 0.0001 BTC                                  |
| Estimated BTC Transacted | 1.0194 BTC                                  |
| Scripts                  | <a href="#">Show scripts &amp; coinbase</a> |

# blockchain.info (and many other sites)

# The Bitcoin network

# The Bitcoin network

- It is a P2P network, where all nodes are equal.
- It runs over **TCP and has a random topology** - random nodes are peered with random other nodes.
- New nodes can join at any time. (So you can download the Bitcoin client today; spin the computer as a node; participating Bitcoin node - with equal rights and capabilities as the other nodes).
- Network is dynamic, forget non-responding nodes after 3 hours.

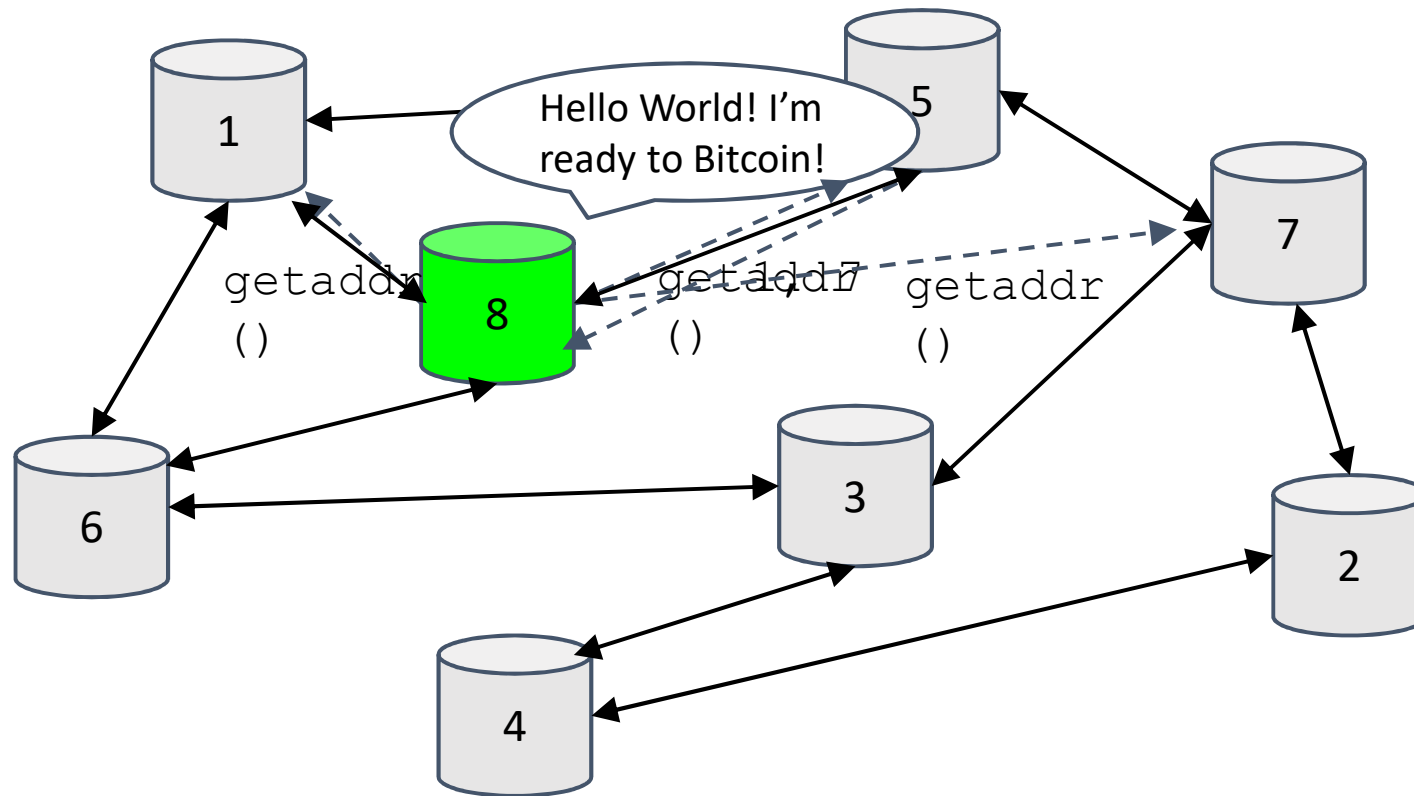
## Bitcoin P2P network

- Ad-hoc protocol (runs on TCP port 8333)
- Ad-hoc network with random topology
- All nodes are equal
- New nodes can join at any time
- Forget non-responding nodes after 3 hr

# Join the Bitcoin P2P network

- Networks connect to other nodes in a random fashion.
- Launch a new node and want to join the network.
  - Start with a simple message to get to one node (seed node) on the network.
  - Find the seed node and send a special message - asking for all the peers of the seed node.
  - The contacted seed node replies with peered nodes.
  - The newly launched node then contacts the seed nodes peers and asking the new seed nodes for their peers.
  - Eventually a list of peers will be gathered by the newly launched node - which then chooses which ones to peer with.
  - Then the **new node will be a fully functional node in the Bitcoin network.**  
(Member of the blockchain);

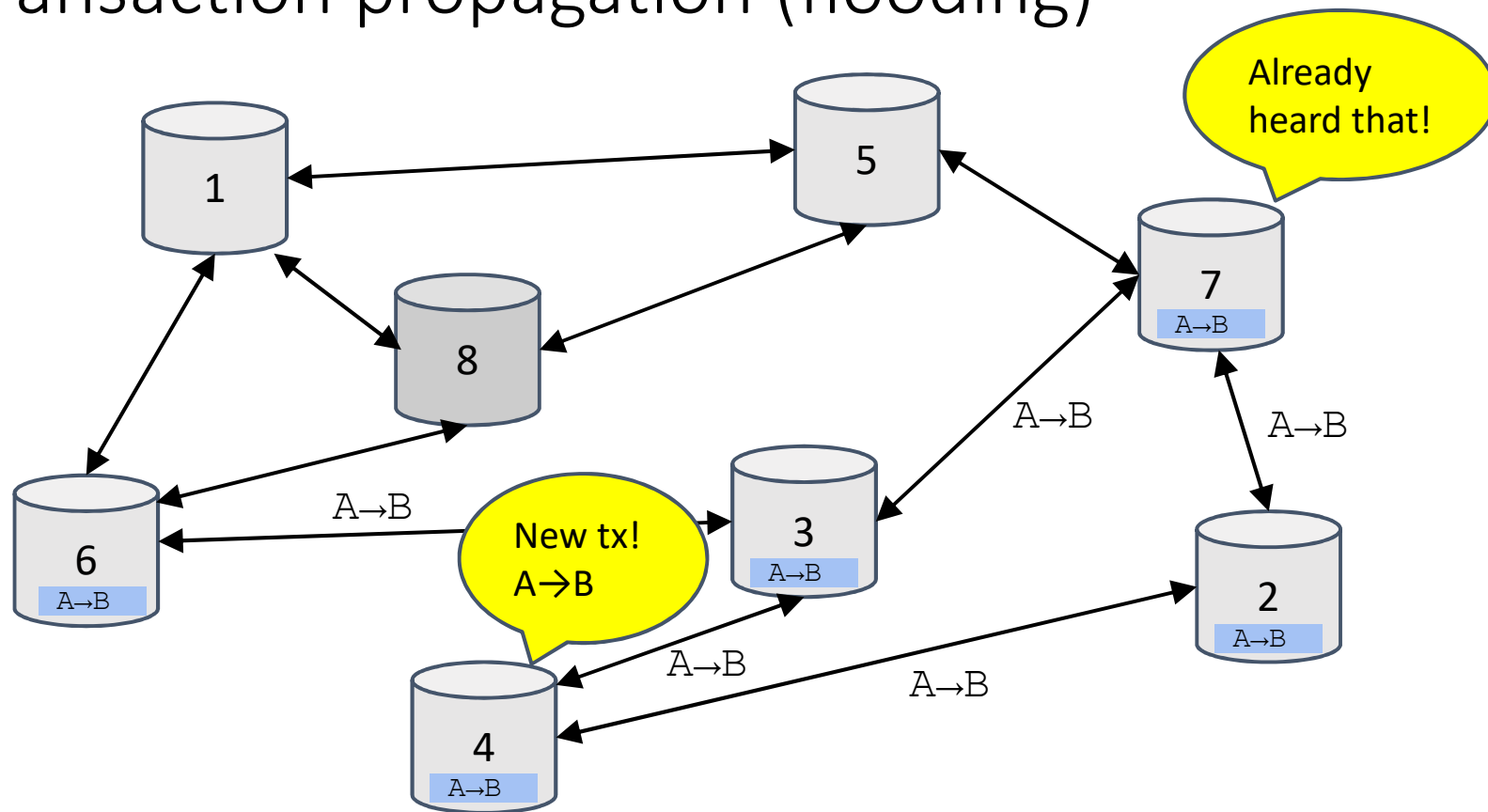
# Joining the Bitcoin P2P network



# Transaction propagation (flooding)

- Network maintains the block-chain, so if any transaction is to be published, the entire network needs to know about it.
- There is a **simple flooding algorithm (sometimes called a gossip protocol)** that takes care of it.
  - Consider node 4, hears about a new transaction (Alice -> Bob);
  - Node 4 communicates the transaction to its peers. Each node maintains a list of all transaction. New transactions are added into the list of the Peer nodes as well.
  - In case peer node as already added the transaction into the list, on listening to the same transaction it will notify the node that it has heard the transaction already (memory pool). (won't loop)
  - Eventually it stops when all the nodes have noted it. Each transaction is uniquely identified by a hash.

# Transaction propagation (flooding)



How do nodes decide when they hear about new transaction whether or not they should propagate it?

- There are four checks.
  - The nodes check if the transaction is valid w.r.t the current block chain.
  - Sanity-checks: (Some nodes may ignore them)
    - (default) script matches a whitelist (Avoid unusual scripts).
    - Haven't seen before conditions (Avoid Infinite loops).
    - Doesn't conflict with others relayed (Avoid double-spend)

# Should I relay a proposed transaction?

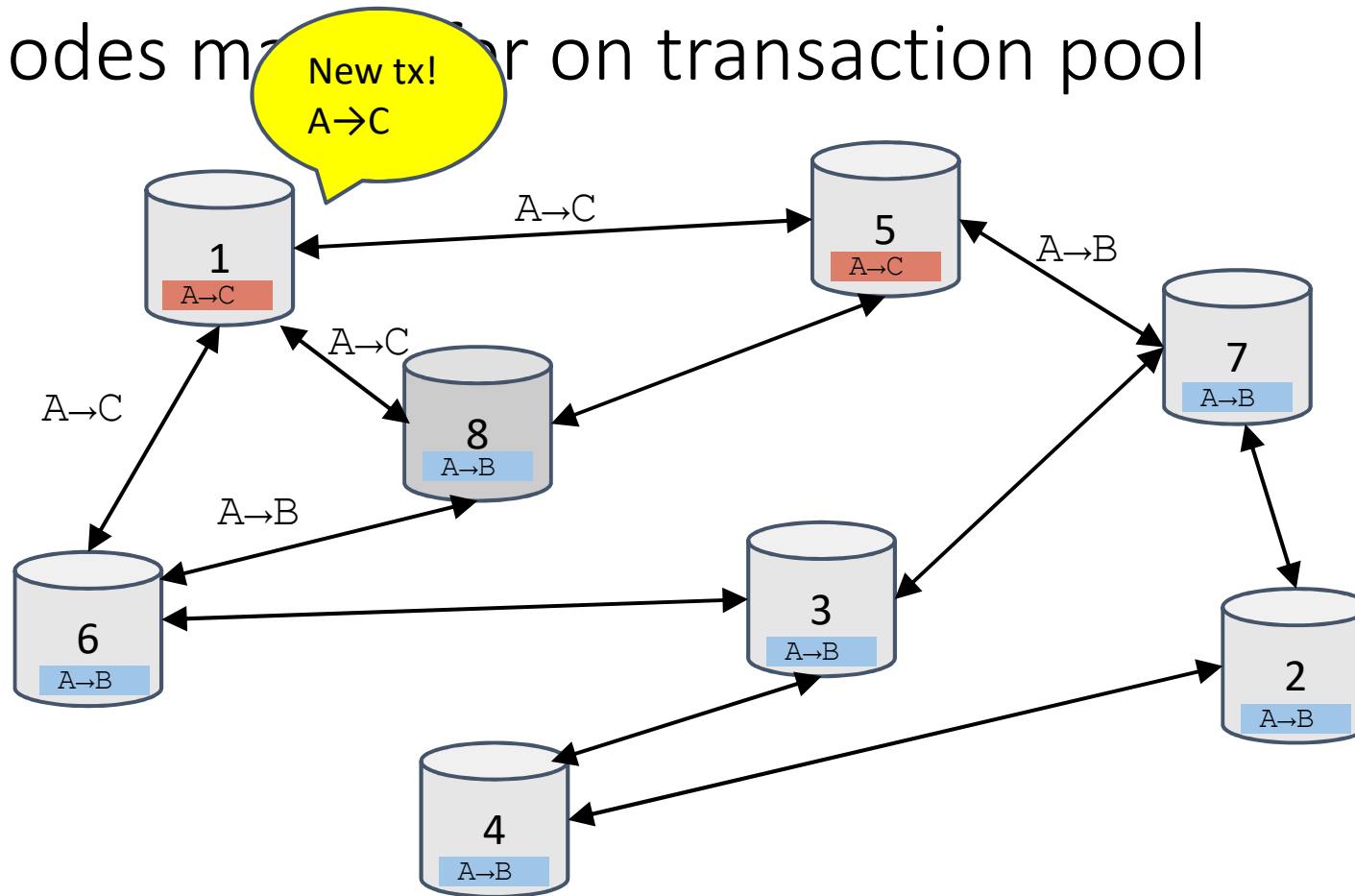
- Transaction valid with current block chain(default)
  - Run script for each previous output being redeemed and ensure that script returns true!
- Script matches a whitelist
  - Avoid unusual scripts
- Haven't seen before
  - Avoid infinite loops
- Doesn't conflict with others I've relayed
  - Avoid double-spends

Sanity checks only...  
Well-behaving nodes implement them!  
Some nodes may ignore them!

# Nodes may differ on transaction pool

- Node 1 - hears a new transaction (Alice -> Charlie), node 1 forwards it to its peers and so on.
- Node 4 - hears a new transaction (Alice -> Bob), node 4 forwards it to its neighbours, etc. (node 1 message hasn't reached node 4);
- Other nodes in between say node 6 between 1 and 4 has **already received message from node 1** and **hence refuses to enlist node 4's message**. (**Conflict**) (**Race condition**).
- Network is in a divided state.
- This is fine, since the transaction has not been published yet - eventually nodes will sort it.

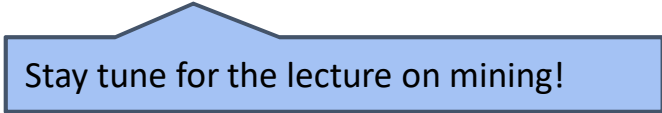
Nodes make a decision on transaction pool



# Race conditions

Transactions or blocks may *conflict*

- Default behavior: accept what you hear first
- Network position matters
- Miners may implement other logic!

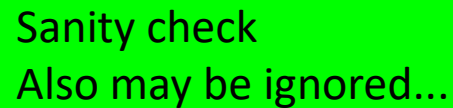


Stay tune for the lecture on mining!

# Block propagation nearly identical

Relay a new block when you hear it if:

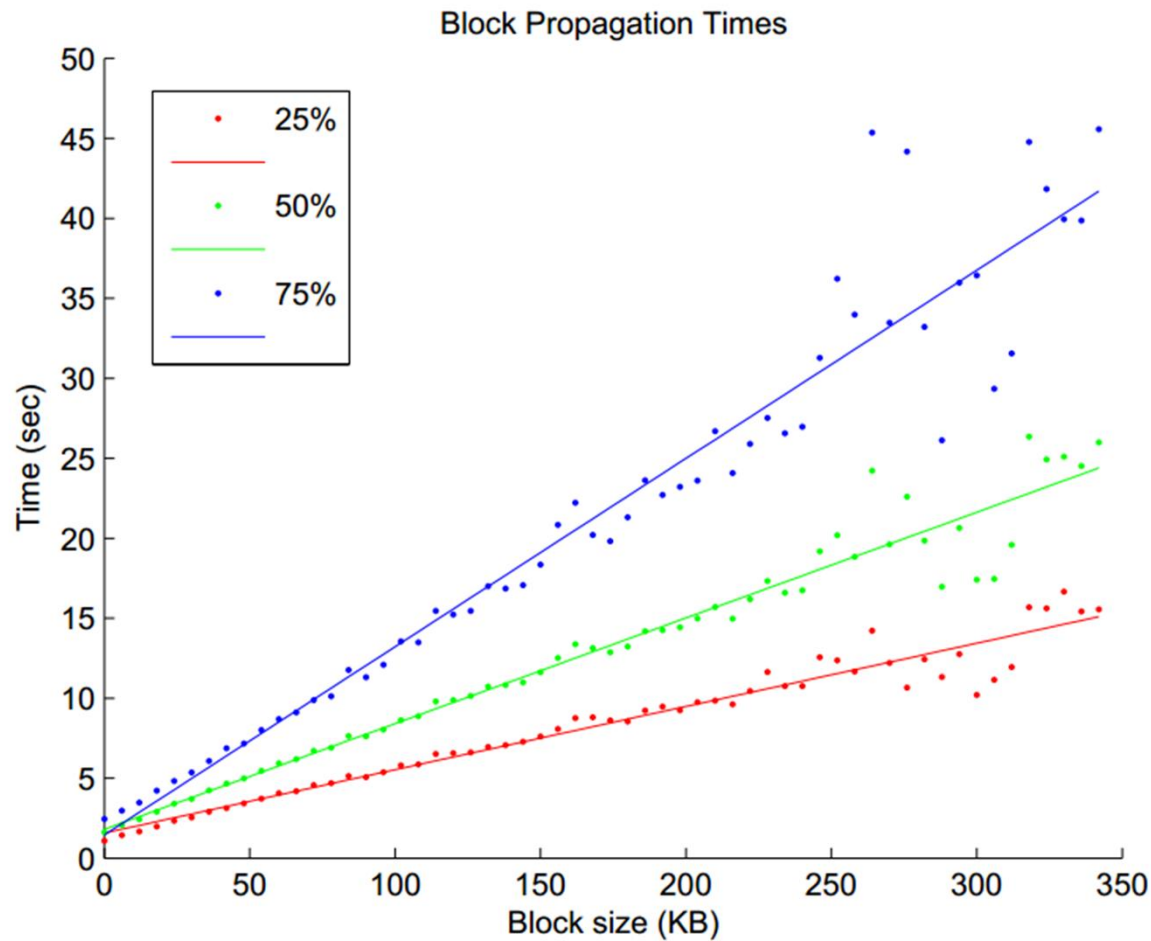
- **Block** meets **the hash target**
- **Block** has all **valid transactions**
  - **Run *all* scripts**, even **if you wouldn't relay**
- **Block builds on current longest chain**
  - Avoid forks



Sanity check  
Also may be ignored...

# Block Propagation time

- Block size (KB) (x-axis) V/s Time (sec) (y-axis)
- 75% - takes > 30sec (long) -> not an efficient algorithm;
- Block needs to reach all nodes.



This graph shows the average time that it takes a block to reach various percentages of the nodes in the network.

Source: Yonatan Sompolsky and Aviv Zohar: "Accelerating Bitcoin's Transaction Processing" 2014

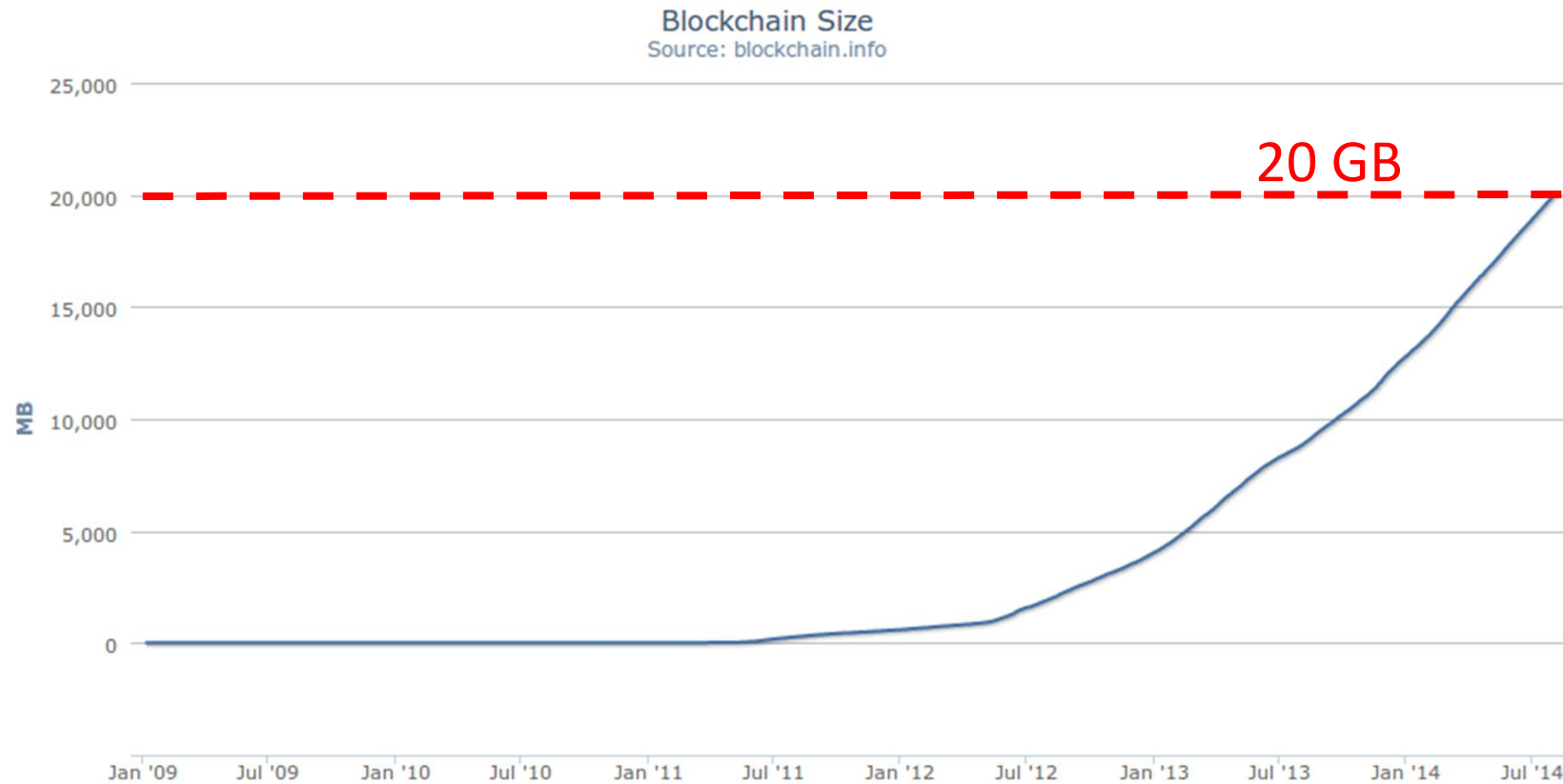
## How big is the network?

- Impossible to measure exactly
- Estimates-up to 1M IP addresses/month
- Only about 5-10k “full nodes”
  - Permanently connected
  - Fully-validated
- This number may be dropping!

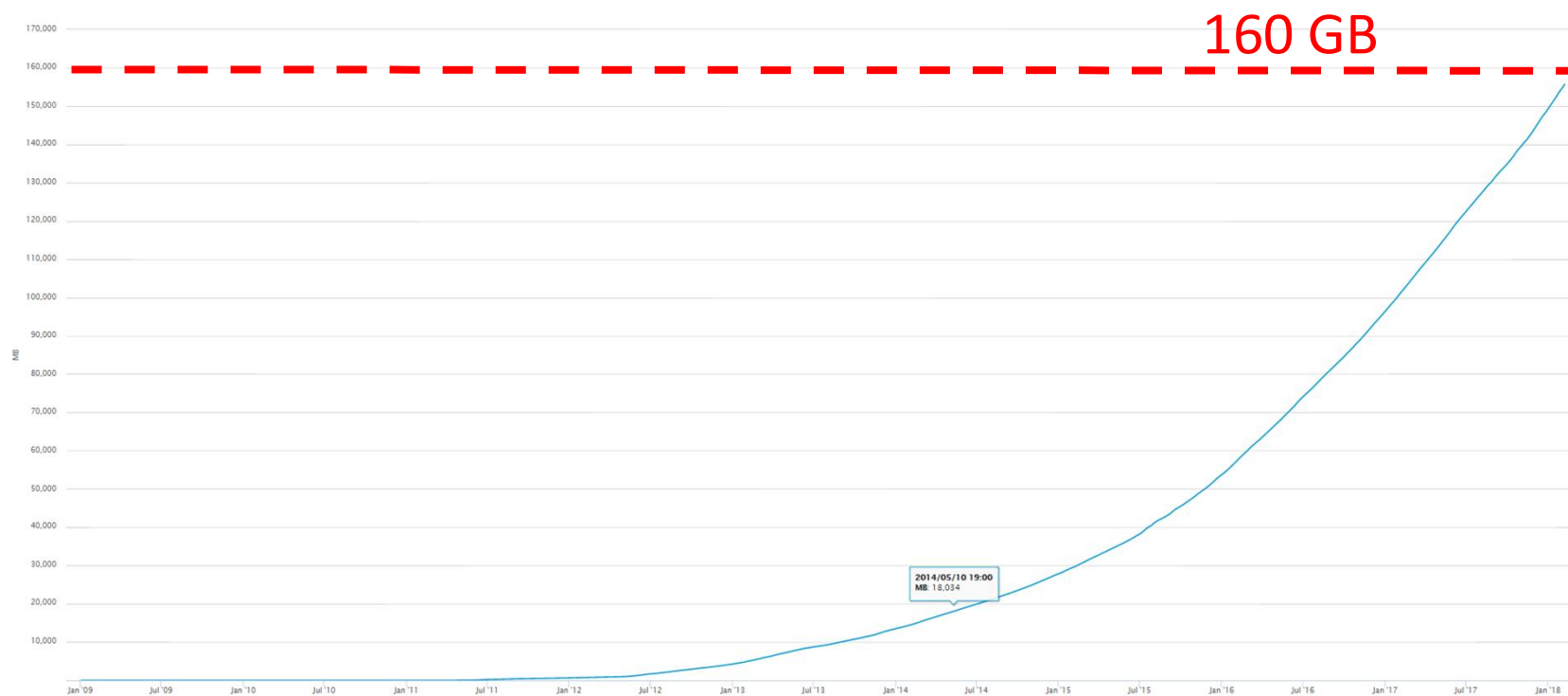
## Fully-validating nodes

- Permanently connected
- Store entire block chain
- Hear and forward every node/transaction

# Storage costs (in 2014)



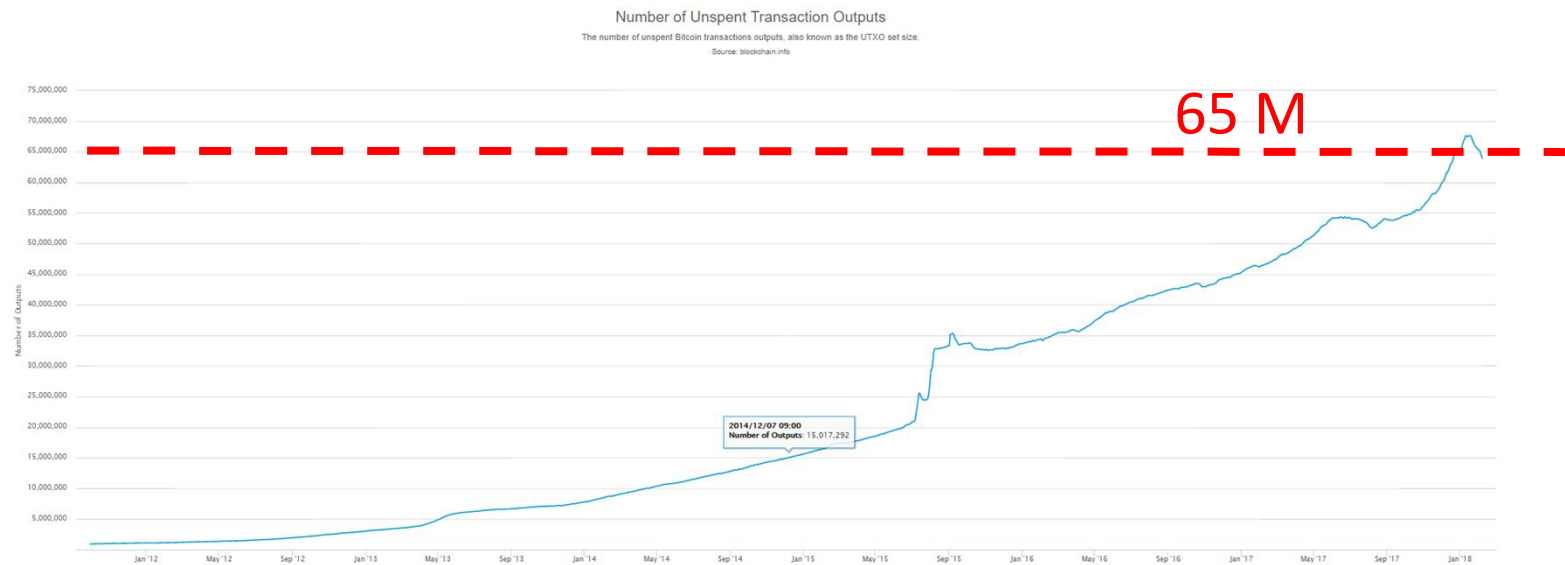
# Storage costs (in 2018)



Source: blockchain.info

# Tracking the UTXO set

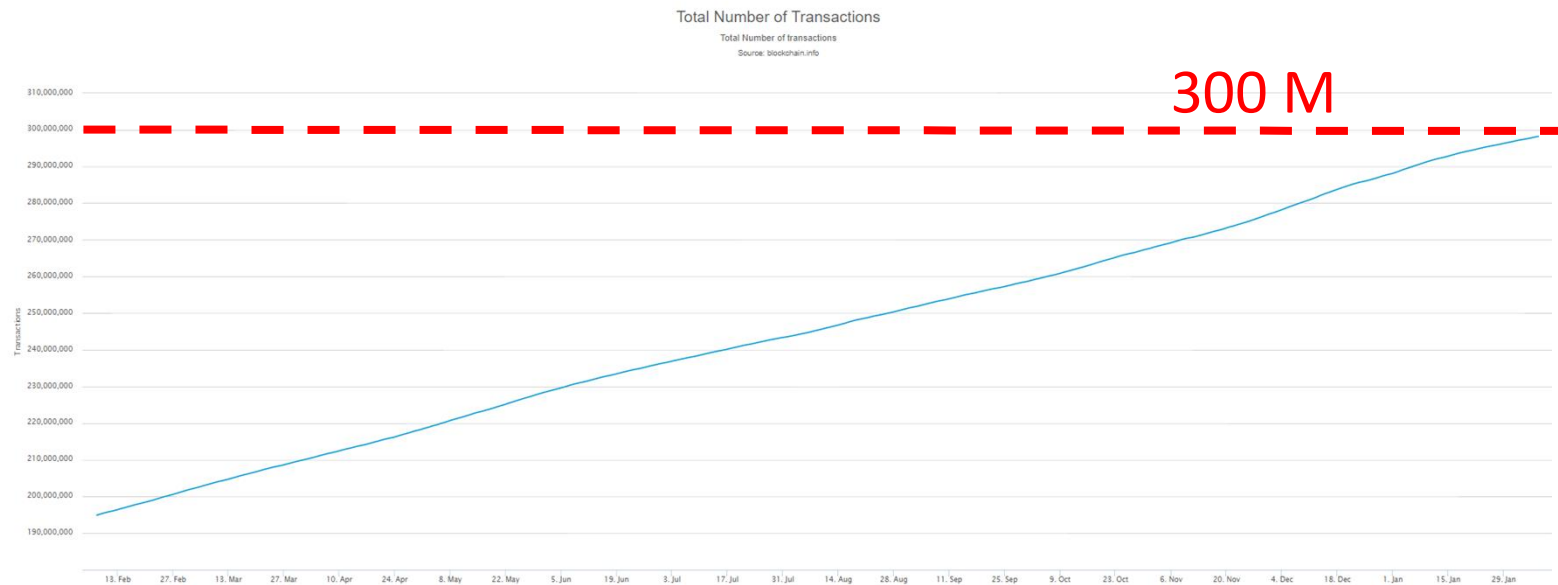
- **U**nspent **T**ransaction **O**utput
  - Should be stored in memory - everything else can be stored on disk, why?



Source: blockchain.info

# Tracking the UTXO set

- Currently ~65 M UTXOs
  - Out of 300 M transactions
- Can require several Gigabytes to store – can it fit in the RAM of a standard computer?



Source: [blockchain.info](https://blockchain.info)

# Thin/SPV clients (not fully-validating)

## SPV or Lightweight nodes

SPV – Simplified Payment Verification (e.g., Wallet nodes)

### Idea: Don't store everything

- Store block headers – **verify the puzzle was solved correctly**, but cannot verify every transaction in each block!
- Validate only those transactions that affect them → By requesting transactions as needed
  - To verify incoming payment
  - Trust fully-validating nodes

1000x cost savings! Requires only a few tens of Megabytes (compare to tens of Gigabytes needed for fully validating nodes)

## Software diversity

- About 90% of nodes run “Core Bitcoin” (C++)
  - Some are out of date versions
- Other implementations running successfully
  - BitcoinJ (Java)
  - Libbitcoin (C++)
  - btcd (Go)
- “Original Satoshi client”

## Limitations & Improvements

## Hard-coded limits in Bitcoin

- 10 min. average creation time per block
- 1 M bytes in a block
- 20,000 signature operations per block
- 100 M *satoshis* per bitcoin
- 23M total bitcoins maximum
- 50,25,12.5... bitcoin mining reward

These affect  
economic  
balance of  
power too  
much to change  
now

## Throughput limits in Bitcoin

- 1 M bytes/block (10 min)
- >250 bytes/transaction
- 7 transactions/sec ☹

Compare to:

- VISA: 2,000-10,000 transactions/sec
- PayPal: 50-100 transaction/sec

## Cryptographic limits in Bitcoin

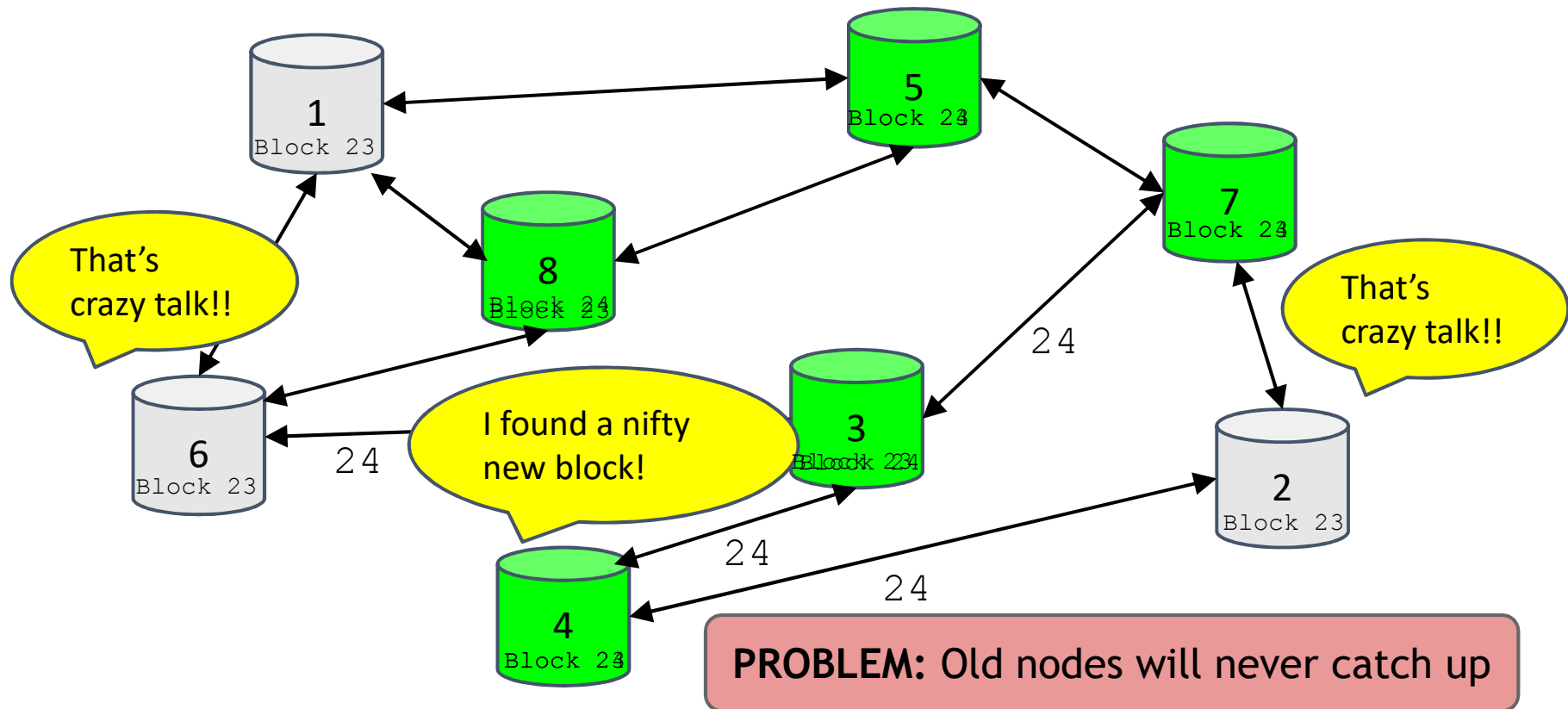
- Only 1 signature algorithm (ECDSA/P256)
- Hard-coded hash functions

Crypto primitives might break by 2040...

# "Hard-forking" changes to Bitcoin

- In case there is a need to upgrade the block-chain.
- It could be possible that not all the nodes are upgraded on time this could lead to issues.
- The upgraded nodes will still continue to process new blocks while the old nodes will be rejecting the new blocks.
- Unfortunately, the old nodes will never catch up.
- It's called "hard-fork" cause the block-chain will split based on which version of the protocol they are running on and will never join again.

# “Hard-forking” changes to Bitcoin



## Soft forks

Observation: we can add new features which only *limit* the set of valid transactions

Need majority of nodes to enforce new rules

Old nodes will approve

**RISK:** Old nodes might mine now-invalid blocks

# Soft forks

- New nodes in the network will be enforcing some new tighter set of rules.
- Relying on enough nodes switching to new version of the protocol that will enable them to enforce new rules knowing that old nodes won't be able to enforce the new rules because they haven't heard them yet.
- Risk: Old nodes might be mining old invalid blocks - because they might be mining valid block but according to the new, strict rules the block is not valid anymore. Eventually the new nodes will reject the block and the old nodes will accept the reject and move on to the next version of the block-chain.(temporary fork)

## Soft fork example: pay to script hash

<signature>  
<<pubkey> OP\_CHECKSIG>

OP\_HASH160  
<hash of redemption script>  
OP\_EQUAL

Old nodes will just approve the hash, not run the embedded script

## Soft fork possibilities

- New signature schemes
- Extra per-block metadata
  - Shove in the coinbase parameter
  - Commit to UTXO tree in each block

## Hard forks

- New op codes
- Changes to size limits
- Changes to mining rate
- Many small bug fixes

Currently seem very unlikely to happen

Stay tuned for the lecture on altcoins!

In the next lecture...

## Human beings aren't Bitcoin nodes

- How do people interact with the network?
- How do people exchange bitcoins for cash?
- How do people securely store bitcoins?

Currency needs to work for people, not nodes